

Automating Alpha: A Quantitative Researcher AI Agent for Systematic Alpha Discovery on WorldQuant Brain

Nuthdanai Wangpratham

Independent Research

February 2026

Abstract. Alpha decay — the systematic degradation of quantitative signal predictive power as markets arbitrage known patterns — compels investment organizations to engage in continuous, high-volume alpha discovery. This process, currently bottlenecked by the cognitive throughput of human quantitative researchers, represents the rate-limiting step of systematic investment management. We present **BrainAlpha**, an end-to-end autonomous multi-agent system that replicates the complete alpha discovery workflow using large language models (LLMs) coordinated through a LangGraph StateGraph pipeline. BrainAlpha comprises six specialized agents (M0–M5) spanning hypothesis ideation, signal specification, expression generation, simulation-based evaluation, evolutionary repair, and portfolio management, all grounded in WorldQuant Brain’s production-grade evaluation infrastructure. Our system establishes a principled framework for replacing the human alpha discovery bottleneck with an automated, scalable, and interpretable LLM-driven research pipeline.

Keywords: quantitative finance, alpha discovery, large language models, multi-agent systems, WorldQuant Brain, LangGraph, automated research

Contents

1	Introduction	4
1.1	The Economics of Quantitative Alpha	4
1.2	The Quantitative Research Process	4
1.3	The Case for Automation: Large Language Models as Quantitative Researchers	6
1.4	WorldQuant Brain as the Research Infrastructure	6
1.5	Research Objective and Contributions	7
1.6	Paper Organization	8
2	Related Work	9
2.1	The Quantitative Alpha Discovery Process	9
2.1.1	Traditional Alpha Mining Paradigms	9
2.1.2	The Alpha Decay Problem and Its Structural Drivers	10
2.1.3	The Formulaic Expression Language	10
2.2	Large Language Models as Financial Analysts	11
2.3	LLM-Based Autonomous Alpha Mining Systems	11
2.3.1	RD-Agent: Data-Centric Factor-Model Co-Optimization	11
2.3.2	Alpha-GPT: Human-AI Interactive Mining with Hierarchical RAG	12
2.3.3	Automate Strategy Finding: Three-Stage LLM and Multimodal Pipeline	12
2.3.4	AlphaAgent: Regularized Exploration via AST Similarity	13
2.3.5	AlphaAgents: Multi-Agent Debate for Portfolio Construction	13
2.3.6	QuantaAlpha: Trajectory-Level Self-Evolution	14
2.4	Research Gaps and Positioning of BrainAlpha	15
3	System Architecture	18
3.1	Design Principles	18
3.2	Pipeline Overview	19
3.3	Module Descriptions	21
3.3.1	M0 — Idea Generation Engine	21
3.3.2	M1 — RAG-Enhanced Seed Factory	22
3.3.3	M2 — Generator Agent	23
3.3.4	M3 — Brain Oracle	24
3.3.5	M3.5 — Error Recovery Agent	25
3.3.6	M4 — Evolution Engine	26
3.4	Shared State Schema: The SignalAlphaObject	28
3.5	Failure Mode Taxonomy	30
3.6	System-Level Properties	32
4	Methodology	33
4.1	M0: Priority-Weighted Structured Exploration	33
4.1.1	Exploration Grid Construction	33
4.1.2	Cell Priority Scoring	34
4.1.3	Parallel Agent Execution	35
4.1.4	Two-Layer Semantic Deduplication	35
4.2	M1: Retrieval-Augmented Specification	35

4.2.1	Vector Store Construction	35
4.2.2	Cosine Similarity Retrieval	36
4.2.3	Four-Stage Processing Pipeline	36
4.3	M2: Chain-of-Thought Expression Generation	37
4.3.1	Five-Step Reasoning Chain	37
4.3.2	AST Diversity Gate	38
4.3.3	Complexity Cap	38
4.4	M3: Brain-Native Evaluation	38
4.4.1	API Interaction and Credential Chain	38
4.4.2	Metrics Parsing with Safe Fallbacks	38
4.4.3	Five-Gate Pass/Fail Judge	39
4.4.4	Failure Mode Classifier and Priority Ordering	40
4.5	M3.5: Error Recovery Agent	40
4.5.1	Trigger Condition and Motivation	40
4.5.2	Three-Strategy Repair Sequence	40
4.5.3	Parallel Batch Execution	41
4.5.4	LangGraph State Machine	41
4.6	M4: Failure-Mode-Aware Iterative Repair	42
4.6.1	Phase 1: Recoverability Assessment	42
4.6.2	Repair Strategy Catalogue	42
4.6.3	LLM Rewriter	43
4.6.4	Iteration Memory and the RepairHistory Object	44
4.6.5	Five Stop Conditions	45
4.6.6	Root Cause Analyzer	45
4.7	Prompt Engineering Methodology	46
5	Experiments and Results	48
5.1	Experimental Setup	48
5.2	Evaluation Metrics	49
5.3	Exploration Coverage and Hypothesis Diversity	50
5.3.1	Grid Coverage	50
5.3.2	Hypothesis Deduplication	51
5.4	Pipeline Throughput and Quota Efficiency	51
5.4.1	Primary Pipeline Cycle (M0-Generated Expressions)	51
5.4.2	M4 Repair Loop Validation	52
5.4.3	Signal Structure Validation (Post-Diagnosis)	53
5.4.4	PV-Family Deep-Dive: 10-Cell Automated Pipeline Run	54
5.5	Signal Quality of Passing Alphas	56
5.6	Ablation Study	57
5.7	Failure Mode Analysis	58
5.7.1	Observed Failure Mode Distribution	58
5.7.2	The Rank Centering Defect: Discovery and Correction	59
5.8	Limitations	60
5.9	Data Availability	61

6	Conclusion and Future Work	62
6.1	Summary of Contributions	62
6.2	Broader Implications	63
6.3	Limitations	64
6.4	Future Work	65
6.5	Closing Remarks	66

“The market is a device for transferring money from the impatient to the patient.”

— Warren Buffett

1 Introduction

1.1 The Economics of Quantitative Alpha

Financial markets are, at their core, competitive information-processing systems. Prices aggregate the collective beliefs of millions of participants, each acting on private signals, public information, and behavioral heuristics. Within this ecosystem, a *quantitative alpha* — a systematic, rules-based signal that predicts future asset returns above a passive benchmark — represents a temporary informational advantage: evidence that some dimension of market prices is not yet fully arbitrated away.

The theoretical foundation for such opportunities is well-established. While the Efficient Market Hypothesis [Fama, 1970] posits that prices reflect all available information, decades of empirical research have documented persistent, replicable return anomalies — momentum [Jegadeesh and Titman, 1993], value [Fama and French, 1992], quality [Novy-Marx, 2013], low volatility [Frazzini and Pedersen, 2014], and hundreds of derivatives thereof [Harvey et al., 2016, Hou et al., 2020]. These anomalies arise from structural market frictions, investor behavioral biases, and institutional constraints that prevent immediate arbitrage. Crucially, they do not persist indefinitely.

Alpha decays. As a signal becomes known — through academic publication, practitioner adoption, or increased capital allocation — the anomaly it exploits is progressively arbitrated away. Empirical estimates place the average half-life of a quantitative alpha in U.S. equities at approximately 5–7 years, with a measurable decay rate of 5.6% per year [McLean and Pontiff, 2016], accelerating to 9.9% per year in European markets. The consequence is unambiguous: a quantitative investment organization that ceases to discover new alphas is, by definition, operating a depreciating asset base. *Alpha discovery is not a one-time research problem — it is a perpetual operational imperative.*

1.2 The Quantitative Research Process

To appreciate why alpha discovery constitutes the central bottleneck of quantitative investment management, it is instructive to map the complete workflow of a professional quantitative research organization. The process, as practiced at leading systematic investment firms, comprises five sequential stages.

Stage 1. Universe and Data Acquisition. Researchers define the investment universe (e.g., U.S. equities, Russell 3000) and assemble a data platform spanning price and

volume history, fundamental financial statements, alternative data sources, and sentiment signals. This stage is largely infrastructure — costly to build but stable once established.

Stage 2. Alpha Discovery (*the critical bottleneck*). Given the data universe, researchers generate, formalize, and test hypotheses about which observable variables predict future returns. A hypothesis begins as an economic intuition and must be translated into a precise, evaluable formulaic expression. For WorldQuant Brain, this means a cross-sectional operator expression such as `rank(ts_delta(earnings, 5))` that can be submitted to a simulation engine for objective evaluation. Each expression is evaluated against a battery of quality checks: minimum Sharpe ratio, fitness score, turnover constraints, drawdown limits, and self-correlation thresholds.

Stage 3. Portfolio Construction. Surviving alphas are combined into a multi-factor portfolio. Combination weights account for factor correlations, capacity constraints, and regime sensitivity. A single high-quality alpha rarely constitutes a complete strategy; portfolio-level diversification across uncorrelated factors is the mechanism by which individual signal noise is suppressed.

Stage 4. Risk Management. The combined portfolio is subjected to risk attribution, factor exposure analysis, and stress testing. Position-level constraints, sector concentration limits, and tail-risk hedging strategies are applied to ensure acceptable behavior under adverse market regimes.

Stage 5. Execution and Monitoring. Approved strategies are deployed in live markets. Ongoing monitoring tracks rolling signal quality metrics — particularly the Information Coefficient (IC) and IC Information Ratio (ICIR) — and triggers strategy retirement when performance degrades below production thresholds ($IC < 0.02$, $ICIR < 0.3$ by standard practice [Grinold and Kahn, 1999]).

Of these five stages, only **Stage 2 requires continuous, high-volume human creative effort**. Stages 1, 3, 4, and 5 are largely amenable to systematic automation once alpha inputs are defined. Stage 2, by contrast, depends on the intersection of financial domain knowledge, mathematical intuition, and empirical creativity — capabilities that have historically been exclusively human. It is also the most time-consuming: a skilled quantitative researcher may generate 2–5 viable alpha ideas per week, of which 15–30% pass initial quality screening [Tang et al., 2025].

The arithmetic is sobering. A research organization seeking to maintain a live portfolio of 1,000 uncorrelated alpha signals must replace roughly 56–100 decaying factors per year at current decay rates, while simultaneously expanding the portfolio to capture new market phenomena. At a generation rate of 2–5 ideas per researcher-week with a 20% hit rate, maintaining this portfolio requires continuous effort from tens of dedicated researchers. **Alpha discovery is the rate-limiting step of quantitative investment management.**

1.3 The Case for Automation: Large Language Models as Quantitative Researchers

The emergence of large language models (LLMs) with sophisticated financial reasoning capabilities presents, for the first time, a credible path toward automating Stage 2 at scale.

LLMs trained on broad corpora of financial literature, academic research, and market commentary have internalized substantial knowledge of return anomalies, factor construction techniques, and empirical asset pricing relationships. Recent empirical evidence confirms that this knowledge is actionable. A landmark study covering 161,000 daily stock-level observations across Russell 1000 constituents demonstrated that frontier LLMs, queried via standard interfaces without fine-tuning, generate stock attractiveness scores that produce a statistically significant daily alpha of **0.185%** ($t = 2.51$) after Fama-French six-factor risk adjustment — approximately **46% in annualized excess return** [Anonymous, 2026]. Five independent statistical tests confirm near-perfect temporal consistency (split-half reliability $\rho = 0.942$) and directional accuracy (rank-return Spearman $\rho = 0.629$).

Building on this foundation, a growing body of research has investigated LLM-based systems specifically designed to generate formulaic alpha expressions in the symbolic language of quantitative platforms. Alpha-GPT [Wang et al., 2025], a human-AI interactive mining system, achieved a **top-10 global ranking** at WorldQuant’s International Quant Championship 2024 among 41,000+ competing teams, delivering IC improvements of 284% over unaided human researchers. AlphaAgent [Tang et al., 2025] introduced principled regularization against alpha decay — enforcing structural novelty via Abstract Syntax Tree (AST) similarity constraints, semantic consistency scoring, and complexity penalization — achieving an **81% improvement in hit ratio** (0.29 vs. 0.16) over the RD-Agent [Microsoft Research Asia, 2025] baseline. Most recently, QuantaAlpha [Han et al., 2026], the current state of the art as of February 2026, applies evolutionary operators at the *trajectory* level to achieve $IC = 0.1501$, $ICIR = 0.9110$, and annualized return of **27.75%** on CSI300, with near-zero-shot transfer to S&P 500 ($\sim 137\%$ cumulative excess return over four years).

The trajectory of these results establishes a clear conclusion: **LLMs are not merely useful tools for quantitative research assistance — they are approaching the capability threshold of autonomous alpha discovery.** The question is no longer whether LLM-based systems can generate viable alphas, but whether they can be engineered into production-grade, end-to-end research pipelines that replicate the full workflow of a human quantitative researcher.

1.4 WorldQuant Brain as the Research Infrastructure

A critical prerequisite for autonomous alpha research is a rigorous, standardized evaluation environment. Backtesting alpha expressions in-house requires substantial data infrastructure, careful survivorship bias controls, realistic transaction cost modeling, and deep expertise in simulation methodology. These requirements represent a significant barrier to rapid, iterative alpha discovery — precisely the workflow that LLM-based systems demand.

WorldQuant Brain addresses this barrier directly. Brain provides a professional-grade simulation platform — the same infrastructure underlying WorldQuant’s live investment strategies — that evaluates submitted alpha expressions against a universe of up to 3,000 stocks with institutional-quality historical data, realistic slippage assumptions, and a standardized battery of quality checks. Critically, Brain imposes *objective, interpretable* pass/fail criteria:

minimum Sharpe ratio thresholds, fitness scores, turnover bounds, drawdown constraints, and self-correlation limits. These structured evaluation signals are deterministic and machine-readable — precisely the type of feedback that LLM-based optimization loops require.

This evaluation infrastructure enables a fundamental separation of concerns: **the LLM agent is responsible exclusively for alpha discovery (the hypothesis-to-expression translation problem), while Brain handles evaluation, simulation, and quality assurance.** The agent need not implement a backtesting engine, manage historical data, or reason about transaction costs — it need only generate expressions in Brain’s well-defined operator vocabulary, interpret structured feedback, and iterate. This architectural separation focuses the entire cognitive burden of the automated system on the task where LLMs add the most value.

1.5 Research Objective and Contributions

This paper presents **BrainAlpha** — an end-to-end autonomous quantitative research system that replicates the complete alpha discovery workflow of a professional quant researcher using a coordinated pipeline of LLM-based agents, retrieval-augmented generation, and structured Brain API integration.

The central thesis is as follows: *if the complete cognitive workflow of alpha discovery — from investment thesis formulation, through signal specification, expression generation, iterative refinement, and failure diagnosis — can be decomposed into well-defined subtasks and assigned to specialized LLM agents, then the rate-limiting bottleneck of quantitative investment management becomes amenable to automation at a scale and cost structure previously unattainable.*

This work makes the following contributions:

- C1. A modular, production-grade multi-agent architecture** for end-to-end alpha discovery, comprising six specialized modules (M0, M1, M2, M3, M3.5, and M4) that collectively replicate the hypothesis-to-portfolio workflow of a human quantitative research team.
- C2. A RAG-enhanced seed factory (M1)** that grounds alpha hypothesis formalization in WorldQuant Brain’s live data universe, eliminating the field-hallucination problem endemic to naive LLM-based expression generation.
- C3. An evolutionary repair engine (M4)** that applies targeted, history-aware LLM rewrites to failing alpha expressions using a structured strategy catalogue, governed by a five-condition stop policy that prevents wasted simulation budget.
- C4. An error recovery layer (M3.5)** that intercepts submission-level failures before they propagate to M4, applying a three-strategy repair sequence (transient retry, conservative LLM fix, creative LLM rewrite) to recover API-error SAOs, and marking irrecoverable candidates with `m4_eligible=False` to prevent wasted simulation budget on structurally broken expressions.
- C5. Empirical evaluation on WorldQuant Brain** — the same production-grade platform used by WorldQuant’s global research organization — providing an externally valid, institutionally grounded benchmark for autonomous alpha system performance.
- C6. A unified taxonomy of LLM-based alpha system failure modes** (FM-1 through FM-6), derived from Brain’s native check structure, enabling systematic failure diagnosis and targeted recovery strategies.

1.6 Paper Organization

The remainder of this paper is organized as follows. Section 2 surveys related work in LLM-based alpha mining, automated hypothesis generation, and quantitative research automation. Section 3 presents the BrainAlpha system architecture, including the M3.5 Error Recovery Agent introduced in this work. Section 4 details the methodology of each module. Section 5 reports experimental results on WorldQuant Brain. Section 6 concludes with implications for the future of automated quantitative research.

“If I have seen further than others, it is by standing upon the shoulders of giants.”

— Isaac Newton

2 Related Work

This section situates BrainAlpha within the broader landscape of quantitative research automation. We proceed in four stages: §2.1 reviews the classical paradigms of human-driven and algorithmic alpha discovery together with the structural forces that make continuous pipeline renewal operationally mandatory; §2.2 reviews direct empirical evidence that LLMs generate actionable financial signals, establishing the capability foundation on which all downstream systems rest; §2.3 profiles six LLM-based autonomous alpha mining systems that constitute the direct prior art for BrainAlpha; and §2.4 identifies the specific research gaps that none of these systems resolve, and maps each gap to a corresponding BrainAlpha contribution.

2.1 The Quantitative Alpha Discovery Process

2.1.1 Traditional Alpha Mining Paradigms

The quantitative investment literature identifies two classical paradigms for generating systematic return signals [Wang et al., 2025].

Paradigm 1: Manual mining. Under this paradigm, a skilled quantitative researcher translates an economic or behavioral hypothesis into a formalized signal by hand. The process is inherently talent-intensive: researchers must simultaneously command empirical asset pricing theory (to generate plausible hypotheses), programming fluency (to implement expressions), and statistical discipline (to avoid in-sample overfitting). A productive researcher generates 2–5 viable ideas per week [Tang et al., 2025], of which typically 15–30% survive initial quality screening. At this throughput, maintaining a large, diversified portfolio of live signals requires a substantial and continuously expanding research team.

Paradigm 2: Algorithmic mining. Genetic programming and related evolutionary search methods [Koza, 1992] attempt to relax the human throughput bottleneck by searching the combinatorial space of formula expressions automatically. A typed grammar defines the vocabulary of operators and operands; a fitness function evaluates candidate expressions against historical data; evolutionary selection pressure gradually improves population quality. While algorithmic mining dramatically increases evaluation throughput, it suffers from three well-documented limitations: the absence of economic intuition means the search space is effectively unbounded and largely unprofitable; the generated expressions are frequently syntactically complex but semantically vacuous; and the approach produces no natural language rationale that researchers can audit or stakeholders can interpret. These limitations motivate the third paradigm — human-AI interactive mining [Wang et al., 2025] — in which a large language model supplies the economic reasoning that genetic algorithms lack, while automated evaluation provides the scalable feedback loop that human researchers cannot sustain.

2.1.2 The Alpha Decay Problem and Its Structural Drivers

The temporal fragility of quantitative signals is the central economic reality that mandates continuous alpha discovery. McLean and Pontiff [2016] provide the canonical empirical characterization: analysing 97 published anomalies across the U.S. equity market, they document an average out-of-sample decay rate of **5.6% per year** in the United States, accelerating to **9.9% per year** in European markets as post-publication arbitrage capital increases. Of the 97 anomalies studied, more than 50 were statistically indistinguishable from zero five years after initial publication — not because the original findings were spurious, but because informed capital rapidly eliminated the mispricing.

The structural drivers of alpha decay operate through three interlocking channels. *Information diffusion*: once a predictive relationship is documented in the academic or practitioner literature, it becomes available to all participants simultaneously, removing the informational asymmetry that originally sustained the signal. *Factor crowding*: as capital allocation to a known factor grows, the arbitrage demand it generates moves prices in a direction that gradually erodes the forward-looking predictability the factor originally exhibited. *Market microstructure adaptation*: high-frequency intermediaries and exchange mechanisms adapt to predictable order flow, effectively front-running systematic strategies and compressing available returns.

The practical implication for a systematic investment organization is unambiguous: maintaining a live portfolio of N uncorrelated signals requires replacing approximately $0.056 \times N$ factors per year in the United States — roughly 56 factors annually for a 1,000-signal portfolio — while simultaneously expanding capacity to capture newly emerging market phenomena. No finite research team can sustain this throughput indefinitely. **Automated alpha discovery is not a research curiosity — it is the operational response to a quantifiable throughput deficit.**

2.1.3 The Formulaic Expression Language

A prerequisite for automated alpha generation is a formal, machine-parsable representation of alpha expressions that permits both systematic generation and deterministic evaluation. Kakushadze [2016] provide this foundation through the *Alpha101* operator vocabulary: a standardized set of cross-sectional and time-series operators — `rank`, `ts_rank`, `delta`, `correlation`, `stddev`, `decay_linear`, `delay`, `signedpower`, `adv{d}` — from which virtually all formulaic signals used in professional practice can be constructed. This operator vocabulary defines a context-free grammar over which LLM-based generation and genetic operators can be applied compositionally.

WorldQuant Brain implements this vocabulary as its native evaluation substrate. Submitted expressions are compiled to a simulation engine that evaluates cross-sectional return predictions against a universe of up to 3,000 stocks, returning structured, machine-readable quality metrics: Sharpe ratio, fitness score, turnover, drawdown, and self-correlation. The determinism and interpretability of this feedback channel is precisely what makes Brain suitable as the evaluation oracle for an autonomous LLM research pipeline.

2.2 Large Language Models as Financial Analysts

Before surveying LLM-based alpha mining *systems*, it is essential to establish the empirical foundation: do LLMs possess genuine, actionable financial knowledge? The evidence now available provides a strongly affirmative answer.

LLMs encode financial knowledge through pretraining. Lopez-Lira and Tang [2023] demonstrate that ChatGPT sentiment scores derived from news headlines predict intraday stock returns with statistical significance, providing early evidence that LLMs internalize the causal structure of information-to-price transmission. Liu et al. [2023] extend this finding through FinGPT, an open-source framework for fine-tuning Llama-class models on curated financial corpora, demonstrating that domain adaptation amplifies performance on downstream tasks including sentiment classification, financial question answering, and portfolio ranking. Multi-agent financial trading frameworks [Zhang et al., 2024] have further demonstrated that coordinated LLM agents can replicate the deliberative reasoning process of professional investment committees, producing signals that outperform individual model baselines.

Direct evidence: agentic LLMs as autonomous equity analysts [Anonymous, 2026]. The most compelling empirical evidence to date is provided by a fully agentic architecture in which a frontier LLM is tasked with rating every Russell 1000 stock daily, using only real-time web search with no curated data inputs and no look-ahead bias by construction. Evaluated over 161,000 stock-day observations from April 2025 through January 2026, the system’s top-20 value-weighted portfolio achieved approximately **50% cumulative return** versus a 26% benchmark return over the same period. After Fama-French five-factor plus momentum risk adjustment, the daily alpha reaches **0.185%** ($t = 2.51$), corresponding to approximately **46% annualized excess return**. The annualized Sharpe ratio reaches **2.43**, a level that would be competitive with institutional quantitative strategies. Five independent statistical tests confirm near-perfect temporal consistency (split-half reliability $\rho = 0.942$) and directional accuracy (rank-return Spearman $\rho = 0.629$). Importantly, the system exhibits an asymmetric capability profile: LLMs are substantially more effective at identifying top-quintile winners than at screening losers, a finding with direct implications for long-only portfolio construction.

These results collectively establish a critical baseline: **LLMs are not merely useful for financial research assistance — they are capable of generating statistically significant alpha from first principles, using only the language capabilities acquired during pretraining.** The remaining challenge is not whether LLMs can generate alpha, but how to engineer these capabilities into a systematic, scalable, production-grade pipeline with principled failure handling and external evaluation grounding.

2.3 LLM-Based Autonomous Alpha Mining Systems

We now survey the six published LLM-based autonomous alpha mining systems that constitute the direct prior art for BrainAlpha, followed by a comparative summary table.

2.3.1 RD-Agent: Data-Centric Factor–Model Co-Optimization

Microsoft Research Asia [2025] introduce RD-Agent, an open-source system from Microsoft Research Asia that frames quantitative alpha mining as a continuous data-centric co-optimization problem. The system coordinates two classes of agents — *Research* agents that propose factor

hypotheses and *Development* agents that implement and evaluate them — in an iterative loop that jointly optimizes factor quality and the predictive model consuming them. The data-centric design philosophy treats factor discovery and model selection as inseparable: the marginal value of a new factor is evaluated in the context of the factors already in the portfolio, incentivizing diversification. RD-Agent provides a functional end-to-end pipeline and serves as the common baseline against which more recent systems report improvements. Its primary limitation, documented empirically by subsequent work, is the absence of principled mechanisms against alpha decay: the system generates factors without constraints on structural similarity to existing signals, leading to a low effective factor ratio (the fraction of generated factors that survive diversity-based quality screening). AlphaAgent reports an **81% improvement** in effective factor ratio relative to RD-Agent, and QuantaAlpha demonstrates consistent further gains.

2.3.2 Alpha-GPT: Human-AI Interactive Mining with Hierarchical RAG

Wang et al. [2025] present Alpha-GPT, published at EMNLP 2025 System Demonstrations, which introduces a three-paradigm taxonomy of alpha mining workflows and positions LLM-based systems as *Paradigm 3: interactive human-AI co-creation*. The system’s agentic pipeline comprises three roles: a *Trading Idea Polisher* (ideation agent), a *Quant Developer* (implementation agent), and an *Analyst* (evaluation and review agent). The system’s most technically distinctive component is a **four-level hierarchical RAG architecture** (RAG#0 through RAG#3) that retrieves contextually relevant alpha templates, data field documentation, prior performance records, and operator-level implementation patterns across a corpus of over 10^4 data fields. An additional *Alpha Search Enhancement* module applies genetic programming operators for iterative refinement of candidate expressions. The system is implemented on Llama-3-70B with BGE-M3 embeddings.

Alpha-GPT’s empirical results are striking: operating on the WorldQuant Brain platform, the system achieved a **top-10 global ranking** at the WorldQuant International Quant Championship 2024, among 41,000+ competing teams, delivering an IC improvement of **284%** over unaided human researchers in the autonomous mode. The primary limitation is that the system’s highest-performance operating mode requires human feedback at each iteration; the autonomous mode, while strong, sacrifices some signal quality.

2.3.3 Automate Strategy Finding: Three-Stage LLM and Multimodal Pipeline

Kou et al. [2025] present a three-component architecture from HKUST that introduces multimodal evaluation capabilities into the alpha mining pipeline. The first component, the *LLM-based Seed Alpha Factory*, uses GPT-4o to generate 100 candidate alpha expressions across 9 thematic categories, providing diverse ideation at scale. The second component, *Multimodal Multi-Agent Evaluation*, deploys two specialized evaluation agents: a *Confidence Score Agent* (CSA) that estimates the statistical robustness of candidate signals from textual and numerical inputs, and a *Risk Preference Agent* (RPA) that adjusts ranking according to a configurable risk tolerance profile. A distinguishing feature is the system’s multimodal input capacity: the evaluation agents accept text, numerical data, audio, video, and visual inputs, enabling a broader class of data sources than systems limited to structured data. The third component applies MLP-based dynamic weight optimization to combine surviving alphas into a final portfolio.

Evaluated on the SSE50 index (January 2023 – January 2024), the system achieves a **cumulative return of 53.17%** against all benchmark strategies, the highest absolute performance reported among systems with a custom backtesting environment. The critical limitation for our purposes is that the system evaluates against a custom backtester on Chinese equities; it is not grounded in a production-grade evaluation oracle, making its results difficult to compare rigorously against Brain-native systems.

2.3.4 AlphaAgent: Regularized Exploration via AST Similarity

Tang et al. [2025] present AlphaAgent, published at KDD 2025, which makes the most technically sophisticated contribution to date on the alpha decay problem. The system implements a closed-loop pipeline of three agents — *Idea Agent* (hypothesis generation), *Factor Agent* (expression implementation), and *Eval Agent* (quality assessment) — augmented by three orthogonal regularization mechanisms that simultaneously enforce structural novelty, semantic consistency, and expression parsimony.

AST similarity regularization computes pairwise subtree isomorphism between candidate expressions and all live factors, rejecting candidates with structural similarity exceeding a calibrated threshold. **Hypothesis-factor alignment** scores each expression against its generating hypothesis using a learned consistency function $C(h, d, f) \in [0, 1]$, penalizing expressions that do not reflect the economic intuition motivating them. **Complexity control** jointly penalizes symbolic expression length and parameter count, biasing the search toward parsimonious signals that generalize more reliably out-of-sample. The composite regularization objective $R_g(f, h) = \alpha_1 \cdot \text{SL}(f) + \alpha_2 \cdot \text{PC}(f) + \alpha_3 \cdot \text{ER}(f, h)$ governs factor acceptance throughout the mining loop.

Evaluated on CSI500 and S&P 500 over January 2021 through December 2024, AlphaAgent achieves **11.0% annualized active return** with $\text{IR} = 1.5$ on CSI500 and **8.74%** ($\text{IR} = 1.05$) on S&P 500, while generating factors with an **81% higher effective factor ratio** and **30% fewer LLM tokens** than RD-Agent. AlphaAgent’s limitation is the absence of structured failure diagnosis: when an expression fails evaluation, the system retries without categorizing the failure mode or adapting strategy based on failure history.

2.3.5 AlphaAgents: Multi-Agent Debate for Portfolio Construction

Zhao et al. [2025], from BlackRock’s quantitative research group, present AlphaAgents, a multi-agent debate architecture for equity portfolio construction rather than formulaic expression generation. Three role-based agents operate concurrently: a *Fundamental Agent* that analyses 10-K and 10-Q filings, a *Sentiment Agent* that processes news and earnings call transcripts, and a *Valuation Agent* that interprets price and volatility time series. The three agents engage in a round-robin debate mechanism, mediated by a coordinator, until consensus is reached on a BUY/SELL/HOLD recommendation for each stock. The system is built on Microsoft AutoGen with GPT-4o and incorporates risk tolerance profiling via prompt engineering, enabling a configurable trade-off between alpha seeking and drawdown aversion.

AlphaAgents is notable for its institutional origin and rigorous multi-modal deliberation mechanism. Its limitation, relevant to the BrainAlpha comparison, is scope: the system produces stock selection signals rather than formulaic alpha expressions, and operates outside the WorldQuant Brain evaluation infrastructure. It does not address the expression generation,

syntax validation, or structured failure recovery challenges that constitute the core technical problems in Brain-native alpha mining.

2.3.6 QuantaAlpha: Trajectory-Level Self-Evolution

Han et al. [2026] present QuantaAlpha (arXiv, February 2026), the current state of the art as of the writing of this paper. The system’s core innovation is a reformulation of the alpha mining problem as *trajectory-level optimization*: each mining run is represented as a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_n)$, where states encode the accumulated alpha portfolio and actions encode individual generation decisions. This framing enables the application of evolutionary operators — mutation and crossover — not to individual expressions (as in genetic programming) but to entire research trajectories.

The architecture comprises four components: *Diversified Planning Initialization* generates structurally diverse starting hypotheses to prevent trajectory collapse into local optima; a *Factor Realization* module with a constraint gate rejects syntactically or semantically malformed expressions before simulation; *Self-Evolution* applies mutation (localizing and rewriting the failure-causing step of a trajectory via self-reflection) and crossover (recombining complementary high-reward trajectory segments); and a *Final Factor Pool* module applies portfolio-level diversity selection. A consistency verifier enforces alignment between the natural language hypothesis, the formal expression, and the implementation code at each step.

QuantaAlpha reports the strongest quantitative results in the field: on CSI300, **IC = 0.1501**, **ICIR = 0.9110**, **annualized return of 27.75%**, and **maximum drawdown of 7.98%**. Out-of-distribution transfer is equally compelling: **160%** cumulative excess return on CSI500 and **137%** on S&P 500 over four years. The system’s acknowledged limitations are stochastic controllability (the trajectory mutation process does not guarantee monotone improvement) and the absence of per-failure diagnostic granularity: self-reflection localizes the failure step but does not classify it into a structured taxonomy that would enable targeted repair strategy selection.

Comparative Summary

Table 1 summarizes the six systems across architecturally and empirically comparable dimensions.

Table 1: Comparative summary of LLM-based autonomous alpha mining systems. [†]Custom backtester; not directly comparable to Brain-evaluated results.

System	Year	Venue	Key Mechanism	Best Reported Metric	Primary Limitation
RD-Agent	2025	Open-source	Data-centric multi-agent factor loop	Common baseline	No decay regularization; low effective factor ratio
Alpha-GPT	2025	EMNLP	4-level hierarchical RAG; genetic refinement	Top-10 IQC (41k+ teams); +284% IC	Interactive mode requires human feedback
Automate	2025	arXiv	3-stage + multimodal agents	53.17% return on SSE50 [†]	Custom backtester; not Brain-native
AlphaAgent	2025	KDD	AST similarity regularization	+81% hit ratio; 11.0% AAR (IR = 1.5)	No structured failure taxonomy or repair
AlphaAgents	2025	arXiv	Multi-agent debate; role specialization	Portfolio construction signal	No formulaic expression generation
QuantaAlpha	2026	arXiv	Trajectory-level mutation/crossover	IC = 0.1501; ARR = 27.75%	Stochastic; no per-failure repair strategy

2.4 Research Gaps and Positioning of BrainAlpha

The six systems surveyed in §2.3 collectively advance the state of the art substantially. Nevertheless, a careful examination of the literature reveals five structural gaps that none of these systems address, and which BrainAlpha is specifically designed to resolve.

Gap 1: No published system targets the WorldQuant Brain production platform directly. All six surveyed systems evaluate against either proprietary backtesting environments (Automate, AlphaAgents) or standard historical data benchmarks on CSI300/S&P 500 (RD-Agent, AlphaAgent, QuantaAlpha). Alpha-GPT is the sole exception: it submits to Brain as part of the IQC competition, but does not expose Brain’s structured API feedback as a first-class signal in its generation loop. As a result, none of the existing systems exploits the full richness of Brain’s quality check output — Sharpe, fitness, turnover, drawdown, and self-correlation scores — as a structured, machine-readable feedback channel for adaptive generation. BrainAlpha addresses this gap through the *M3 Brain Oracle* module, which wraps the Brain API, parses all returned quality metrics, and exposes them as typed state variables in the LangGraph pipeline (Contribution C4).

Gap 2: No system provides a unified taxonomy of failure modes. Existing systems treat evaluation failure as a binary signal: a candidate expression either passes or fails, and the response is to retry. None of the surveyed work provides a principled decomposition of the failure space into distinct, diagnosable modes. This is a significant omission: the appropriate corrective action differs fundamentally depending on *why* an expression fails. A signal failing on turnover requires a different repair strategy than one failing on self-correlation, which in turn

requires a different approach than one failing on fitness. BrainAlpha addresses this gap through a **unified six-class failure taxonomy** (FM-1 through FM-6) derived from Brain’s native check structure, enabling the M4 Evolution Engine to select targeted repair strategies from a structured catalogue rather than issuing undifferentiated retry prompts (Contribution C5).

Gap 3: Human involvement remains either explicit or the agent loop is unobservable. Alpha-GPT’s highest-performing operating mode explicitly includes a human feedback stage. RD-Agent and AlphaAgent lack structured observability: the agent loop state is not persisted or inspectable beyond logged outputs, making diagnosis and debugging of long-running runs difficult. QuantaAlpha’s trajectory structure improves on this, but the system does not expose intermediate state in a form amenable to production monitoring. BrainAlpha resolves both dimensions through **LangGraph StateGraph**: the complete M0–M5 pipeline state is persisted at every node transition, enabling real-time observability, deterministic replay, and checkpoint-based recovery from any failure point (Contribution C1).

Gap 4: Field hallucination in expression generation is unaddressed. A fundamental failure mode of LLM-based expression generation is the hallucination of data field names that do not exist in the target platform’s data universe. An expression referencing `earnings_surprise_q` may be syntactically valid and economically motivated but will fail evaluation because the field does not exist in Brain’s data catalogue. Among the surveyed systems, only Alpha-GPT’s RAG architecture partially addresses this: its RAG#1 level retrieves relevant data fields before generation. However, this retrieval is not specifically grounded in the live Brain data universe; it operates over a general financial field corpus. BrainAlpha addresses this gap through the **M1 RAG-enhanced Seed Factory**, which indexes Brain’s live, versioned data field catalogue and enforces field validity at the hypothesis specification stage, before expression generation begins (Contribution C2).

Gap 5: Trajectory evolution has not been applied to per-failure targeted repair. QuantaAlpha’s trajectory-level mutation is the most sophisticated exploration mechanism in the current literature: it localizes the failure-causing step in a mining trajectory and rewrites only that segment, preserving successful upstream steps. However, this mutation is applied at the trajectory level and does not incorporate the diagnostic information available from Brain’s structured check output. The repair is guided by self-reflection over the trajectory history, not by a failure-mode-aware strategy catalogue. BrainAlpha addresses this gap through the **M4 Evolution Engine**, which combines trajectory-history awareness with explicit failure classification: each failing expression is first diagnosed via the FM taxonomy, and the selected repair strategy is drawn from a structured catalogue indexed by failure class. A five-condition stop policy (expression budget, time budget, consecutive failure threshold, self-correlation saturation, and IC floor) governs the repair loop, preventing unbounded simulation expenditure on unrecoverable candidates (Contribution C3).

Positioning. BrainAlpha occupies a position in the design space that is not merely incrementally different from prior work, but structurally distinct along the most practically important axis: institutional deployment readiness. It is the only system that (i) targets the WorldQuant Brain production platform as its native evaluation oracle, (ii) exposes a fully observable, stateful LangGraph pipeline amenable to production monitoring and replay, (iii) provides a unified failure taxonomy enabling systematic repair, and (iv) grounds expression generation in the live Brain

data universe to eliminate field hallucination. In combination, these properties constitute not merely another LLM-alpha system, but the first institutionally grounded research infrastructure designed for deployment within a professional quantitative investment organization.

“Simplicity is the ultimate sophistication.”

— Leonardo da Vinci

3 System Architecture

This section presents the BrainAlpha system architecture: the structural decisions, data contracts, and inter-module routing that together constitute a production-grade autonomous alpha research pipeline. We begin with the four design principles that constrain every architectural choice (§3.1), then describe the pipeline topology (§3.2), detail each of the five production modules (§3.3), define the shared state schema that binds them (§3.4), formalize the failure mode taxonomy central to adaptive repair (§3.5), and conclude with a discussion of the system-level properties that distinguish BrainAlpha as institutionally deployable (§3.6).

3.1 Design Principles

The BrainAlpha architecture is governed by four design principles, each of which resolves one or more of the research gaps identified in §2.4.

P1. Separation of Concerns. The system partitions the alpha discovery workflow into two fundamentally different classes of responsibility. The LLM subsystem — modules M0 through M2, M3.5, and M4 — is responsible exclusively for *creativity*: generating investment theses, translating them into formal specifications, constructing alpha expressions in Brain’s operator vocabulary, and repairing broken or failing expressions. The Brain WebSim API — module M3 — is responsible exclusively for *truth*: evaluating candidate expressions against live historical data under production-grade simulation conditions. This separation is not merely an organizational convention; it is an architectural guarantee. The LLM subsystem never backtests its own outputs. The evaluation oracle never modifies generation logic. Each subsystem can evolve independently, and the quality of the feedback signal is not compromised by the generation mechanism that produced the candidate.

P2. Stateful Observability. All pipeline state is persisted at every node transition in the LangGraph `StateGraph`. At any point in a mining run, the complete history of any candidate alpha — its originating thesis, specification decisions, expression, simulation metrics, repair iterations, and stop condition — is recoverable from the graph checkpoint store without re-execution. This property is not a debugging convenience; it is a production requirement. A pipeline that runs autonomously for hours across potentially hundreds of Brain simulation calls must be deterministically replayable from any checkpoint in the event of infrastructure failure. Stateful observability also enables asynchronous human review without interrupting pipeline execution — a property that Alpha-GPT’s interactive mode requires at the cost of throughput. BrainAlpha achieves equivalent transparency without halting the loop (addresses Gap 3 from §2.4).

P3. Structured Feedback over Binary Retry. When a candidate expression fails Brain’s quality checks, the system does not issue an undifferentiated retry. Instead, the Brain Oracle (M3) classifies the failure into one of six diagnostic categories (FM-1 through FM-6, defined in §3.5), and the Evolution Engine (M4) selects a repair strategy from a structured catalogue indexed by failure class. The fundamental insight motivating this principle is that the failure space is not homogeneous: a signal failing on turnover is a qualitatively different diagnosis than one failing on drawdown or self-correlation, and each failure class has a distinct set of actionable corrections. Treating all failures identically — as all prior systems do — is equivalent to treating a patient with the same medication regardless of the diagnosis. Structured feedback converts the evaluation oracle’s binary output into a rich, typed diagnostic signal (addresses Gap 2).

P4. Data-Grounded Generation. The generative path from investment thesis to formal alpha expression is grounded at two levels. First, the RAG-Enhanced Seed Factory (M1) retrieves verified data field names from Brain’s live field catalogue before any expression is generated, ensuring that the specification encodes only fields that the simulation engine can evaluate. Second, the Generator Agent (M2) constructs expressions using only the operator vocabulary documented in Brain’s implementation guide, enforced via a syntax validator at the output boundary. Data-grounded generation eliminates the field-hallucination failure mode — in which a syntactically plausible expression references a data field that does not exist in the target universe — at the earliest possible stage of the pipeline, before any simulation budget is consumed (addresses Gap 4).

3.2 Pipeline Overview

BrainAlpha is implemented as a directed graph with conditional routing, not a linear chain. The distinction is architecturally material: a linear chain processes each candidate once and discards it; a directed graph with feedback loops enables iterative repair, adaptive routing based on intermediate results, and principled termination when repair is exhausted.

The six production modules — M0, M1, M2, M3, M3.5, and M4 — are arranged in the following topology.

Forward path. The primary forward path proceeds as:

$$M0 \longrightarrow M1 \longrightarrow M2 \longrightarrow M3$$

M0 selects an unexplored cell from the structured exploration grid and produces a **ThesisObject** encoding the investment hypothesis. M1 grounds this thesis in Brain’s data universe via retrieval-augmented generation, producing a validated **SA0** with hypothesis and specification fields populated. M2 generates a formulaic alpha expression from the specification, enforces structural diversity and syntax validity, and attaches the expression to the **SA0**. M3 submits the expression to Brain’s WebSim API, parses the returned quality metrics (both in-sample and out-of-sample), and routes based on the simulation outcome.

Conditional routing at M3. The Brain Oracle (M3) produces one of three outcomes and routes accordingly:

- **PASS:** all quality gates are satisfied. The SAO is written to the passing-alpha output log with `status="pass"` and `m4_eligible=True`.
- **FAIL:** Brain returns simulation metrics, but one or more quality gates fail. The Oracle classifies the failure mode and routes the annotated SAO to the Evolution Engine (M4) for repair.
- **ERROR:** the Brain WebSim submission itself fails (API timeout, authentication error, malformed expression that Brain cannot compile). No simulation metrics are returned; the SAO carries `status="error"` and is routed to the Error Recovery Agent (M3.5).

$$M3 \xrightarrow{\text{PASS}} \text{Output} \quad M3 \xrightarrow{\text{FAIL}} M4 \quad M3 \xrightarrow{\text{ERROR}} M3.5$$

Error recovery (M3.5). The Error Recovery Agent intercepts SAOs with `status="error"` before they reach M4. M3.5 applies up to three escalating repair strategies (transient retry, conservative LLM fix, creative LLM rewrite) and re-submits through M3’s full evaluation pipeline after each attempt. If the expression becomes submittable (producing any simulation result, including a FAIL), it is considered *recovered* and forwarded with `m4_eligible=True`:

$$M3.5 \xrightarrow{\text{RECOVERED (pass)}} \text{Output} \quad M3.5 \xrightarrow{\text{RECOVERED (fail)}} M4 \quad M3.5 \xrightarrow{\text{ABANDONED}} M0$$

If all three strategies fail, the SAO is marked `status="abandoned"` with `m4_eligible=False` and returned to M0.

Repair loop (M4). The Evolution Engine (M4) attempts to repair failing candidates — from both M3 (direct fail) and M3.5 (recovered fail) — through a bounded iterative loop. At each iteration, M4 selects a repair strategy from the failure-class catalogue, invokes a targeted LLM rewrite, and routes the repaired candidate back to M3 for re-evaluation. If the repaired candidate passes, it exits to the output log (`status="repaired"`). If M4 exhausts its repair budget or reaches any of the five stop conditions defined in §3.3.6, it generates a **StopReport** and routes control back to M0:

$$M4 \xrightarrow{\text{PASS}} \text{Output} \quad M4 \xrightarrow{\text{STOP}} M0$$

The M4-to-M0 feedback arc is the mechanism by which the system retires exhausted exploration cells and advances to structurally distinct regions of the alpha hypothesis space. The **StopReport** produced at loop termination provides M0 with a diagnostic summary of which cell was retired and why, enabling informed selection of the next exploration target.

Complexity. The full pipeline implements a *directed graph with two output sinks* (passing alpha output and stop-triggered M0 restart), *one fault-recovery branch* (M3.5 error recovery), and *one conditional repair cycle* (M3→M4→M3 improvement loop). This is the minimum topology necessary to support iterative repair with principled termination and systematic error recovery; any simpler graph would require either unbounded retry or immediate discard of submission failures.

3.3 Module Descriptions

3.3.1 M0 — Idea Generation Engine

The Idea Generation Engine is responsible for the highest-level creative act in the pipeline: deciding *what to investigate*. Its design is motivated by a fundamental inefficiency in prior systems: without a structured exploration map, an LLM prompted to “generate a quantitative alpha hypothesis” will repeatedly return variations of the same momentum or value themes that dominate its pretraining corpus. This is not a capability limitation — it is a sampling problem. The hypothesis space is vast, but an unguided LLM explores only the high-probability region near the centre of its prior.

Structured exploration grid. M0 organizes the hypothesis space into a **660-cell structured exploration grid**, defined as the Cartesian product of four categorical dimensions:

- (i) **Signal Family** (11 classes, matching Brain’s native `category_id` taxonomy): price/volume (pv), fundamental, analyst, earnings, model-based, news, sentiment, risk, options, macro, and alternative.
- (ii) **Operator Pattern** (5 classes): TS_MOM (time-series momentum), TS_REV (time-series reversion), CS_RANK (cross-sectional rank), RATIO (ratio/composite), and COND (conditional/regime).
- (iii) **Time Horizon** (3 classes): short (SH: 1–5 days), medium (MD: 5–20 days), and long (LG: 20–120 days).
- (iv) **Conditioning Regime** (4 classes): unconditional (UNC), liquidity-conditioned (LIQ), volatility-conditioned (VOLA), and sector-conditioned (SECT).

Each cell in the $11 \times 5 \times 3 \times 4 = 660$ -cell grid represents a structurally distinct hypothesis subspace. The 11-family taxonomy is deliberately aligned with Brain’s native `category_id` field, ensuring that every generated alpha is immediately classifiable within Brain’s own organizational structure and that exploration coverage maps directly onto Brain’s signal universe. The grid is the exploration map: cells are marked *active*, *exhausted*, or *reserved*. An exhausted cell — one for which M4 has issued a **StopReport** without producing a passing alpha — is not re-entered until the full grid has been swept, preventing the pipeline from spending disproportionate budget on locally difficult regions.

Instruction Agent. An Instruction Agent (meta-agent) maintains the global exploration state and assigns cells to sub-agents. The Instruction Agent implements a priority queue over active cells, weighted by estimated cell productivity (based on the fraction of passing alphas generated from each signal family and operator pattern in prior runs) and by the recency of the last cell visit. This assignment mechanism enforces *structural deduplication at the planning stage*: because two sub-agents are never assigned overlapping cells simultaneously, the system avoids semantic duplication between concurrently executing hypothesis threads. This is qualitatively superior to *reactive* deduplication (checking a new hypothesis against a database of existing ones after generation), because it prevents redundant generation effort rather than discarding it after the fact.

Output. M0 produces a **ThesisObject**: a structured record containing the assigned cell coordinates (signal family, operator pattern, time horizon, conditioning regime), a natural language investment thesis, a causal mechanism statement (why this signal should predict returns), and a data domain hint (which data categories are likely relevant). The **ThesisObject** is the seed from which all downstream modules derive their inputs.

3.3.2 M1 — RAG-Enhanced Seed Factory

The RAG-Enhanced Seed Factory translates the abstract investment thesis produced by M0 into a precise, machine-evaluable signal specification. This translation is the step at which the hallucination failure mode is most dangerous: an LLM tasked with specifying a signal in terms of concrete data fields will, if ungrounded, invent field names that sound plausible but do not exist in the target data universe. M1’s architecture is specifically designed to eliminate this failure mode before any simulation budget is consumed.

Dual knowledge base. M1 maintains two retrieval-augmented knowledge bases:

KB-1. Brain Live Field Catalogue (KB-1). A vector index over WorldQuant Brain’s complete data field catalogue, ingested from Brain’s `data.csv` manifest. Each document encodes a data field name, its category, description, coverage universe, and temporal availability. KB-1 is versioned and refreshed on a scheduled basis to track Brain’s evolving data offerings. Retrieval from KB-1 returns the top- k fields most semantically relevant to the investment thesis, *all of which are confirmed to exist* in the live evaluation environment.

KB-2. Operator Vocabulary (KB-2). A vector index over Brain’s operator documentation, ingested from the `Operator.txt` reference. Each document encodes an operator name, its mathematical definition, argument types, return type, and a set of canonical usage examples. Retrieval from KB-2 returns the operators most semantically aligned with the signal family and operator pattern coordinates from M0’s cell assignment.

Processing pipeline. M1 processes the **ThesisObject** through four sequential stages:

- (1) **Thesis Polisher.** An LLM pass refines the raw thesis into a structured hypothesis statement aligned with the Alpha101 vocabulary [Kakushadze, 2016], making the retrieval queries more precise.
- (2) **RAG Retrieval.** Parallel retrieval from KB-1 and KB-2 returns k_1 verified data fields and k_2 candidate operators relevant to the polished hypothesis. Retrieval results are ranked by semantic similarity to the hypothesis embedding.
- (3) **Structured Encoder.** An LLM pass, provided with the polished hypothesis and the retrieval results, populates the **SPECIFICATION** fields of the **SA0**: the specific data field names to use, the operators to apply, the investment universe, and the neutralization scheme. Critically, *only fields returned by KB-1 retrieval are permitted in the specification*; the encoder is explicitly instructed to select from the retrieved field list rather than generating field names freely.

- (4) **SAO Validator.** A deterministic validator checks that all data fields in the specification are present in the KB-1 index, that all operators are present in the KB-2 index, and that the specification is internally consistent (e.g., the neutralization scheme is compatible with the investment universe). If validation fails, M1 iterates back to the Structured Encoder with an error annotation.

Output. M1 produces a partial SAO with the HYPOTHESIS and SPECIFICATION sections populated and the EXPRESSION section empty. Field hallucination is structurally impossible at this stage: every data field name in the specification is a confirmed entry in Brain’s live field catalogue.

3.3.3 M2 — Generator Agent

The Generator Agent is responsible for the core intellectual translation of the pipeline: converting the structured specification produced by M1 into a syntactically valid, structurally diverse alpha expression in Brain’s operator vocabulary.

Chain-of-thought generation. M2 employs a five-step chain-of-thought reasoning protocol:

- Step 1. Mechanism.** Articulate the economic mechanism: why should the specified data fields predict future cross-sectional returns? Which direction is the predicted effect, and over what time horizon?
- Step 2. Data.** Identify the primary data field(s) from the specification and reason about their statistical properties (stationarity, distribution, typical cross-sectional dispersion).
- Step 3. Operators.** Select the appropriate operators from KB-2 retrieval results to implement the hypothesized mechanism. Reason about the order of operator application and parameter selection (e.g., lookback window length relative to the target time horizon).
- Step 4. Normalization.** Determine the appropriate cross-sectional normalization scheme (rank transform, z-score, or standardization) to ensure the expression produces a well-conditioned signal for the simulation engine.
- Step 5. Expression.** Compose the final alpha expression as a single string in Brain’s operator grammar, using only the data fields and operators identified in the preceding steps.

This structured reasoning chain serves two purposes. First, it provides a natural language rationale that is stored alongside the expression in the SAO, enabling post-hoc interpretability of any generated signal. Second, it constrains generation to economically grounded expressions: an LLM that must articulate a mechanism before writing code is substantially less likely to produce syntactically valid but economically vacuous expressions.

AST Diversity Gate. Before accepting a candidate expression, M2 computes its Abstract Syntax Tree (AST) and applies the **AST Diversity Gate**. The expression is parsed into a tree representation, and a Jaccard similarity score is computed between the node multiset of the candidate and the node multiset of every expression already in the current mining run’s portfolio:

$$J(e_{\text{new}}, e_i) = \frac{|N(e_{\text{new}}) \cap N(e_i)|}{|N(e_{\text{new}}) \cup N(e_i)|}$$

where $N(e)$ denotes the multiset of AST node types in expression e . If $\max_i J(e_{\text{new}}, e_i) > \theta$ for threshold $\theta = 0.70$, the candidate is rejected and M2 regenerates with an explicit instruction to use a structurally different composition of operators. This gate operationalizes the anti-decay regularization principle of Tang et al. [2025] within BrainAlpha’s pipeline, enforcing that the generated expression library remains structurally diverse across the full mining run.

Syntax Validator and Complexity Cap. Following the diversity gate, a deterministic syntax validator parses the expression against Brain’s operator grammar to confirm that all function calls are well-formed, argument types are compatible, and field references are present in KB-1. Additionally, a **Complexity Cap** rejects any expression with more than 40 AST nodes. The cap is motivated by the observation that long, deeply nested expressions in the WorldQuant operator vocabulary rarely generalize out-of-sample: complexity-penalized generation has been shown to improve the effective factor ratio by up to 30% in comparable systems [Tang et al., 2025].

Output. M2 produces a fully populated SAO with the `EXPRESSION` section completed and `status="pending"`. The AST hash of the expression is stored to support diversity checking in subsequent iterations.

3.3.4 M3 — Brain Oracle

The Brain Oracle is the system’s interface to the objective evaluation environment. It wraps the WorldQuant Brain WebSim API, submits candidate expressions, parses the structured response, applies the pass/fail decision rule, and — in the event of failure — classifies the failure into the taxonomy defined in §3.5.

API interaction. M3 submits the expression from the SAO’s `EXPRESSION` section to the Brain WebSim endpoint with the configured simulation parameters (universe, region, delay, and decay settings). On success, the response is parsed to extract a comprehensive set of quality metrics from the full Brain alpha dictionary, spanning three categories:

- **In-sample (IS) metrics:** Sharpe ratio, fitness score, daily turnover, maximum drawdown, returns, margin, long/short count, book size, PnL, simulation start/end dates, and Brain’s native `checks[]` validation array.
- **Out-of-sample (OS) metrics:** the same metric set evaluated on a held-out OS period (`os_sharpe`, `os_fitness`, `os_returns`, `os_drawdown`, `os_turnover`, etc.), providing forward-validation evidence without additional simulation cost.
- **Alpha-level metadata:** `alpha_id`, `alpha_status`, `alpha_grade`, `self_correlation`, and `category_id`.

All values are written to the `EVALUATION` section of the SAO before any routing decision is made, ensuring complete metric provenance regardless of outcome. If the submission itself fails (API error, network timeout, or Brain rejection), M3 writes `status="error"` with `m4_eligible=False` and routes the SAO to M3.5.

Pass/Fail Judge. The Judge applies the following five-gate acceptance rule:

G1. Sharpe > 1.25

G2. Fitness > 1.0

G3. Turnover $\in [0.01, 0.70]$

G4. Drawdown < 0.25

G5. Returns > 0

An expression must satisfy all five gates simultaneously to receive `judge_status="pass"`. Failure on any gate triggers FM classification. In addition, all entries in `checks[]` must carry `result="PASS"` (with `SELF_CORRELATION="PENDING"` acceptable during Brain’s asynchronous computation window). The thresholds reflect WorldQuant Brain’s documented production quality standards and are enforced exactly — partial credit does not exist.

FM Classifier. The FM Classifier examines the specific gate(s) that failed and assigns a failure mode code. When multiple gates fail simultaneously, the Classifier applies a priority ordering (FM-1 $\succ$ FM-2 $\succ$ FM-4 $\succ$ FM-5 $\succ$ FM-3 $\succ$ FM-6) to assign a single primary failure mode, because the repair strategies are mutually incompatible and the most structurally severe failure should be addressed first. The complete taxonomy is defined in §3.5.

Routing. Upon classification, M3 writes the assigned failure mode code, a structured `failure_detail` payload, and boolean fail-flag columns (`fail_sharpe`, `fail_fitness`, `fail_turnover`, `fail_drawdown`, `fail_returns`) to the SA0, then routes based on outcome: (i) PASS — writes `status="pass"`, `m4_eligible=True`, routes to the output sink; (ii) FAIL — writes `status="fail"`, `m4_eligible=True`, routes to M4; (iii) ERROR — writes `status="error"`, `m4_eligible=False`, routes to M3.5 for error recovery. In all cases, M3 does not modify the `EXPRESSION` section — it is strictly an observer and classifier, not a generator.

M3 directly addresses Gap 1 (§2.4): it is the first module in any published system to wrap Brain’s production API and expose its full structured output as typed state variables in an LLM orchestration framework, enabling every downstream component to reason about simulation metrics rather than binary outcomes. It directly addresses Gap 2 by providing the failure classification that makes structured repair possible.

3.3.5 M3.5 — Error Recovery Agent

The Error Recovery Agent is a fault-tolerance layer that sits between the Brain Oracle (M3) and the Evolution Engine (M4). It is activated exclusively when a Brain WebSim submission fails with a `status="error"` — a condition in which no simulation metrics are available because the API call itself did not succeed. This is categorically distinct from an expression that simulates successfully but fails Brain’s quality gates; those cases proceed directly to M4.

Motivation. Submission failures arise from transient causes: authentication timeouts, network interruptions, rate-limiting, or malformed expressions that Brain’s parser rejects before simulation. Without M3.5, all such SAOs would be permanently discarded with `m4_eligible=False`, wasting the M0–M2 investment in thesis formulation and expression generation. M3.5 recovers these candidates through a bounded sequence of escalating repair strategies before routing them to M4.

Three-strategy repair loop. M3.5 applies up to three repair strategies in increasing order of aggressiveness:

Attempt 1. Retry-same. The original expression is resubmitted without modification. This handles transient API and network errors, which are the most frequent cause of submission failures.

Attempt 2. Conservative LLM fix ($T = 0.2$). The LLM receives the original expression and the error message and is instructed to make the *minimal change* necessary to enable successful submission: replacing unknown field names, adding `pasteurize()` for NaN/Inf safety, correcting parenthesis imbalance, or substituting unsupported operator calls.

Attempt 3. Creative LLM rewrite ($T = 0.6$). If conservative repair also fails, the LLM is given the full investment context (hypothesis, neutralization scheme, operator family, lookback range) and instructed to write a clean, simple replacement expression that captures the same investment idea using only confirmed-safe base fields (`returns`, `close`, `volume`, `adv20`). Expression fidelity is sacrificed in favour of submission success.

After each repair attempt, the fixed expression is routed through M3’s complete evaluation pipeline. A result with `status` $\neq$ "error" (any Brain simulation response, including a quality-gate FAIL) is treated as *recovered* and the SAO is forwarded with `m4_eligible=True`.

Abandonment. If all three strategies fail, M3.5 sets `status="abandoned"` and `m4_eligible=False`. Abandoned SAOs are not forwarded to M4; they are logged, and the originating cell receives a de-weighting in M0’s priority queue.

Parallel execution. M3.5 processes a batch of error SAOs using a `ThreadPoolExecutor` with `max_workers=3`, matching the maximum repair-attempt count. This collapses the worst-case wall-clock time for a batch from $O(n \times 3)$ sequential Brain calls to $O(n)$ parallel calls, enabling error recovery to complete within the same time budget as the primary M3 evaluation pass.

Output. M3.5 produces a `final_sao` for each input SAO with one of three statuses: "pass" or "fail" (`m4_eligible=True`, full IS and OS metrics populated), or "abandoned" (`m4_eligible=False`). The combined output of M3 (direct pass/fail) and M3.5 (recovered pass/fail) forms the complete input batch for M4.

3.3.6 M4 — Evolution Engine

The Evolution Engine is the system’s adaptive repair mechanism. Its function is to transform a failing candidate alpha into a passing one through a bounded sequence of targeted, history-aware LLM rewrites, without discarding the valid economic insight embedded in the original expression.

Two-phase structure. M4 operates in two sequential phases for each failing candidate:

Phase 1: Recoverability Assessment. Before initiating any repair iteration, the Recoverability Assessor evaluates whether the failing candidate is structurally repairable. It applies a fast kill rule: if the assigned failure mode is FM-1 (syntax error) and the expression has already been regenerated twice without fixing the syntax, the candidate is immediately retired to avoid wasting

simulation budget on malformed expressions. Similarly, if the SAO’s repair history (the `history` field defined in §3.4) shows that the expression has previously failed on the same failure mode across three consecutive iterations with no metric improvement, the Recoverability Assessor classifies the candidate as *structurally unrecoverable* and issues a retirement directive. The Recoverability Assessor thus functions as a fast-fail gate that prevents Phase 2 from processing candidates with negligible recovery probability.

Phase 2: Improvement Loop. If the candidate passes the recoverability assessment, M4 enters the improvement loop. The loop proceeds for at most $T_{\max} = 8$ iterations. At each iteration:

- (1) The **Strategy Selector** retrieves the repair strategy catalogue entry indexed by the current failure mode code (FM-1 through FM-6) and selects the highest-priority untried strategy for this candidate.
- (2) The **Rewrite Agent** applies the selected strategy to the current expression, guided by the chain-of-thought reasoning trace stored in the SAO and the metric values from all prior evaluation attempts. The Rewrite Agent has access to the full `history` field, enabling it to avoid repair directions that have been tried and have failed.
- (3) The repaired expression is validated by M2’s syntax validator and AST diversity gate before being routed to M3 for re-evaluation.
- (4) M3 evaluates the repaired expression and updates the SAO’s `EVALUATION` section. If the expression passes, M4 exits with `status="repaired"`. If it fails, the new failure mode and metrics are appended to `history` and the loop continues.

Stop conditions. The improvement loop is governed by five stop conditions, each of which triggers a different recovery action:

- S1. Budget Exhaustion.** $T_{\max} = 8$ iterations reached without a passing expression.
- S2. Performance Plateau.** The best Sharpe ratio across the last three iterations has not improved by more than $\epsilon = 0.05$, indicating convergence to a local optimum.
- S3. Metric Divergence.** A previously improved metric has regressed below its value at the start of the repair loop, suggesting the repair trajectory is counterproductive.
- S4. Oscillation.** The expression has alternated between the same two failure modes across four consecutive iterations, indicating that repairs for one failure mode are inducing the other.
- S5. Persistent FM-1 Failure.** The expression has failed syntax validation on more than three consecutive repair attempts, suggesting the expression structure is fundamentally incompatible with Brain’s parser.

StopReport and M0 routing. Upon triggering any stop condition, M4’s Root Cause Analyzer generates a **StopReport**: a structured diagnostic object recording the triggered condition, the metric trajectory across all repair iterations, the sequence of failure modes encountered, and a natural language assessment of why the cell failed to produce a passing alpha. The **StopReport** is attached to the SAO and routed to M0, which marks the originating cell as exhausted and

selects the next active cell for exploration. M0’s cell priority weighting is updated based on the **StopReport** contents: cells in the same signal family as an exhausted cell receive a temporary de-weighting to encourage broader exploration.

M4 directly addresses Gap 5 (§2.4): it combines trajectory-history awareness — access to the full sequence of prior repair attempts and their outcomes — with explicit failure classification and a structured per-failure repair strategy catalogue. This combination is absent from all prior systems, including QuantaAlpha, which applies trajectory-level mutation without per-failure diagnostic grounding.

3.4 Shared State Schema: The **SignalAlphaObject**

All modules in the BrainAlpha pipeline communicate exclusively through a single, versioned data contract: the **SignalAlphaObject** (SAO). No module holds private state that is not captured in the SAO; no inter-module communication occurs outside the LangGraph state update mechanism. This design guarantees that the complete history of every candidate alpha — from its originating thesis through every evaluation and repair attempt — is accessible to any module that holds a reference to the SAO.

The SAO is organized into six logical sections corresponding to the pipeline stages that populate them. Table 2 provides the complete field-level specification.

Table 2: SignalAlphaObject (SAO) field schema. Each section is populated by the indicated module(s). OS metrics follow the same schema as IS metrics and are prefixed `os_`.

Section	Field	Type	Set by	Description
1. IDENTITY	<code>id</code>	string	M0	UUID for this candidate
	<code>created_at</code>	datetime	M0	Timestamp of thesis creation
	<code>cell_id</code>	string	M0	Exploration grid cell code
	<code>cell_coords</code>	tuple	M0	(family, pattern, horizon, regime)
2. HYPOTHESIS	<code>thesis</code>	string	M0	Natural language investment thesis
	<code>mechanism</code>	string	M0	Causal mechanism statement
	<code>signal_family</code>	string	M0	Grid dimension: signal family
	<code>time_horizon</code>	string	M0	Grid dimension: time horizon
3. SPECIFICATION	<code>data_fields</code>	list	M1	Verified Brain field names (from KB-1)
	<code>operators</code>	list	M1	Selected operator names (from KB-2)
	<code>universe</code>	string	M1	Investment universe identifier
	<code>neutralization</code>	string	M1	Neutralization scheme
4. EXPRESSION	<code>expression</code>	string	M2	Brain operator expression string
	<code>ast_hash</code>	string	M2	Hash of AST node multiset
	<code>complexity</code>	int	M2	AST node count
	<code>status</code>	string	M2/M3/M3.5/M4	<code>pending</code> / <code>pass</code> / <code>fail</code> / <code>error</code> / <code>abandoned</code> / <code>repaired</code>
5. EVALUATION	<code>alpha_id</code>	string	M3	Brain alpha ID (portal link)
	<code>sharpe</code>	float	M3	IS annualized Sharpe ratio
	<code>fitness</code>	float	M3	IS Brain fitness score
	<code>turnover</code>	float	M3	IS daily turnover fraction
	<code>drawdown</code>	float	M3	IS maximum drawdown
	<code>returns</code>	float	M3	IS annualized returns
	<code>self_correlation</code>	float	M3	Self-correlation with live portfolio
	<code>os_sharpe ... os_pnl</code>	float	M3	Out-of-sample counterparts for all IS metrics
	<code>checks</code>	list	M3	Brain native validation check array
	<code>m4_eligible</code>	bool	M3/M3.5	True iff expression produced a simulation result
6. REPAIR	<code>failure_mode</code>	string	M3	Assigned FM code (FM-1 – FM-6)
	<code>failure_detail</code>	dict	M3	Raw metric values at failure
	<code>fail_sharpe</code>	bool	M3	Sharpe gate failed
	<code>fail_fitness</code>	bool	M3	Fitness gate failed
	<code>fail_turnover</code>	bool	M3	Turnover gate failed
	<code>fail_drawdown</code>	bool	M3	Drawdown gate failed
	<code>fail_returns</code>	bool	M3	Returns gate failed
	<code>iteration</code>	int	M4	Current repair iteration count
	<code>history</code>	list	M4	(expression, metrics, FM) per iteration
	<code>stop_report</code>	dict	M4	StopReport payload on loop exit

The **SAO** serves as a *complete audit trail* for every candidate alpha. The **m4_eligible** flag is the primary routing signal between the evaluation layer (M3/M3.5) and the repair layer (M4): only SAOs with **m4_eligible=True** are forwarded to M4, ensuring that the repair engine never processes candidates for which no simulation feedback exists. The boolean fail-flag columns (**fail_sharpe** through **fail_returns**) encode exactly which quality gates triggered the failure, enabling M4’s Strategy Selector to index the repair catalogue without re-examining raw metric values. For a candidate that passes after repair iterations, the **history** field records the full sequence of (expression, Sharpe, fitness, turnover, drawdown, failure mode) tuples, providing a complete provenance record. For a retired candidate, the **stop_report** captures the diagnostic rationale consumed by M0 for future cell prioritization. The out-of-sample metric fields (**os_sharpe** through **os_pnl**) are populated by M3 at zero additional simulation cost, providing held-out validation evidence alongside every in-sample evaluation.

3.5 Failure Mode Taxonomy

A central architectural contribution of BrainAlpha is the **failure mode taxonomy**: a principled decomposition of the alpha expression failure space into six mutually exclusive, collectively exhaustive diagnostic categories derived from Brain’s native quality check structure. This taxonomy is the formal specification referenced throughout the system; it is the index key of M4’s repair strategy catalogue and the classification schema applied by M3’s FM Classifier.

Table 3 presents the complete taxonomy.

Table 3: BrainAlpha failure mode taxonomy (FM-1 through FM-6). Priority ordering governs multi-gate failure resolution. Repair strategy classes reference M4’s strategy catalogue.

Code	Priority	Brain Check	Diagnosis	Repair Strategy Class
FM-1	1 (highest)	Compilation / parse error	Expression fails Brain’s parser; syntax or field reference invalid	Syntax correction: re-validate field names against KB-1; fix operator argument types
FM-2	2	$\text{Fitness} \leq 0$ or $\text{Sharpe} \leq 0$	Signal has no predictive structure; returns are indistinguishable from noise	Fundamental redesign: respecify data fields or invert the signal direction
FM-3	5	$0 < \text{Sharpe} < 1.25$	Signal is predictive but insufficiently consistent over time	Signal amplification: extend lookback, add volatility scaling, or apply additional filters
FM-4	3	Turnover $\notin [0.01, 0.70]$	Signal rebalances too frequently (high turnover) or too rarely (low turnover)	Turnover regulation: adjust lookback window length or apply <code>decay_linear</code> smoothing
FM-5	4	$\text{Drawdown} \geq 0.25$	Signal exhibits excessive peak-to-trough equity losses	Tail-risk mitigation: add volatility-normalized scaling or introduce regime conditioning
FM-6	6 (lowest)	Self-correlation high	Expression is structurally redundant with existing live signals	Diversification: modify operator composition to increase AST distance from the portfolio

Priority ordering rationale. When multiple gates fail simultaneously, assigning a single primary failure mode requires a principled priority ordering. FM-1 takes absolute priority because a syntax error renders all quality metrics undefined; no meaningful repair strategy can be selected until the expression is parseable. FM-2 takes second priority because zero-fitness candidates have no predictive signal whatsoever — any repair that does not address the fundamental predictive structure is wasted. FM-4 (turnover) and FM-5 (drawdown) take priority over FM-3 (Sharpe) because they are structural properties of the signal’s rebalancing behaviour that typically override Sharpe improvements: a high-turnover fix that reduces Sharpe is still net positive if it brings the expression within the feasible region. FM-6 (self-correlation) takes the lowest priority because it is a portfolio-level property, not an expression-level deficiency; a self-correlated candidate may become acceptable if the live portfolio composition changes.

The taxonomy is intentionally aligned with Brain’s evaluation infrastructure. Each FM code corresponds to one or more of Brain’s native quality checks, ensuring that the diagnostic and repair cycle operates on the same information substrate as the evaluation oracle. This alignment is the mechanism by which BrainAlpha resolves Gap 2 (§2.4): structured failure

diagnosis replaces binary retry.

3.6 System-Level Properties

The architectural choices described in this section collectively produce three system-level properties that distinguish BrainAlpha as a production-grade research infrastructure, as distinct from a research prototype.

Property 1: Full Observability. The LangGraph `StateGraph` implementation persists the complete pipeline state at every node transition via the LangGraph checkpoint store. Any module in the pipeline can be interrogated at any point in a mining run for the current `SA0` of every active candidate. Completed runs can be replayed deterministically from any checkpoint, enabling post-hoc analysis of generation and repair trajectories. This property reduces the operational cost of managing long autonomous runs: failures, anomalies, and unexpectedly successful repair trajectories can all be investigated without re-executing from scratch.

Property 2: Structured Recoverability. BrainAlpha does not fail silently. Every candidate alpha that does not pass Brain’s quality checks is either repaired by M4 or retired with a `StopReport` that documents why repair failed. Every exhausted cell is reflected in M0’s updated cell priority weights. The system thus accumulates *negative knowledge* — structured information about what does not work and why — and uses this information to inform future exploration decisions. This is in contrast to prior systems that discard failing candidates without diagnostic capture.

Property 3: Quota Efficiency. WorldQuant Brain operates under simulation quotas: each submitted expression consumes a finite allocation. BrainAlpha’s architecture is specifically designed to minimize wasteful quota consumption through two mechanisms. First, M4’s Phase 1 Recoverability Assessor fast-kills structurally unrecoverable candidates before any repair iterations are attempted. Second, the five stop conditions govern M4’s Phase 2 improvement loop, preventing unbounded simulation spend on candidates that are converging but cannot cross the pass threshold. Together, these mechanisms ensure that quota is concentrated on candidates with meaningful probability of passing — maximizing the effective alpha discovery rate per unit of simulation budget.

“In God we trust; all others must bring data.”

— W. Edwards Deming

4 Methodology

This chapter descends from the architectural overview of §3 to the algorithmic interior of each module. Where Chapter 3 specifies *what* each module does, Chapter 4 specifies *how*: the exact scoring formulas, retrieval mechanics, prompt designs, repair catalogues, and convergence criteria that determine system behavior at runtime.

Three methodological themes recur across all modules and motivate the specific design choices documented below.

- T1. Structured over free-form.** At every stage where generation is required, the system imposes structural constraints that restrict the LLM’s output space to a tractable, domain-valid subspace. Grid cells constrain hypothesis space (M0); RAG retrieval constrains field names (M1); AST gates constrain expression diversity (M2); a repair catalogue constrains rewrite directions (M4). Unconstrained LLM generation is never deployed in a pipeline position where its failures cannot be caught deterministically downstream.
- T2. Deterministic gates first, LLM second.** Every pipeline stage applies its cheapest valid check before invoking any LLM. The recoverability assessor (M4) applies three hard-stop conditions without a single LLM token before the rewriter is invoked. M3’s judge and M1’s validator are entirely LLM-free. This ordering minimizes token cost and eliminates LLM variance from decisions that are more reliably handled by code.
- T3. Context-rich prompts.** When LLMs are invoked, prompts are dynamically assembled to include all information that a human expert would consult: the retrieved knowledge base slice (M1), the full repair history table (M4), failing check values with explicit gap percentages (M4), and per-agent investment philosophy (M0). The cognitive scaffolding encoded in the prompt is the primary mechanism by which the system elicits expert-level reasoning from a general-purpose language model.

4.1 M0: Priority-Weighted Structured Exploration

4.1.1 Exploration Grid Construction

The hypothesis space is discretized into a 660-cell exploration grid defined as the Cartesian product of four categorical axes:

Axis 1. Signal Family (11 codes, aligned with Brain’s native `category_id` taxonomy): `pv` (price/volume), `fundamental`, `analyst`, `earnings`, `model`, `news`, `sentiment`, `risk`, `option`, `macro`, `alt` (alternative data).

Axis 2. Operator Pattern (5 codes): **TS_MOM** (time-series momentum), **TS_REV** (time-series reversion), **CS_RANK** (cross-sectional rank), **RATIO** (ratio/composite), **COND** (conditional/regime).

Axis 3. Time Horizon (3 codes): **SH** (short: 1–5 days), **MD** (medium: 5–20 days), **LG** (long: 20–120 days).

Axis 4. Conditioning Regime (4 codes): **UNC** (unconditional), **LIQ** (liquidity-conditioned), **VOLA** (volatility-conditioned), **SECT** (sector-conditioned).

The product $11 \times 5 \times 3 \times 4 = 660$ defines the complete grid. A cell is identified by a dot-separated code such as **pv.TS_MOM.SH.UNC**, which unambiguously encodes the (price/volume, time-series momentum, short horizon, unconditional) subspace.

Each cell maintains a lifecycle status drawn from $\{\textit{empty}, \textit{assigned}, \textit{explored}, \textit{exhausted}\}$. A cell transitions from *empty* to *assigned* when an agent is dispatched to it, to *explored* when a **ThesisObject** has been generated, and to *exhausted* when M4 has issued a **StopReport** without producing a passing alpha. Exhausted cells are not revisited until all active cells have been explored, enforcing breadth-first coverage of the hypothesis space.

4.1.2 Cell Priority Scoring

Cell selection is governed by a three-factor weighted priority score:

$$\text{Priority}(c, a) = 0.5 \cdot \text{Novelty}(c) + 0.3 \cdot \text{ExpectedYield}(c) + 0.2 \cdot \text{AgentAlignment}(c, a) \quad (1)$$

where c is the candidate cell and a is the requesting agent.

Novelty(c) is a monotone decreasing function of the cell’s exploration status:

$$\text{Novelty}(c) = \begin{cases} 1.00 & \text{if } c \text{ is } \textit{empty} \\ 0.50 & \text{if } c \text{ is } \textit{assigned} \\ 0.25 & \text{if } c \text{ is } \textit{explored} \\ 0.00 & \text{if } c \text{ is } \textit{exhausted} \end{cases}$$

ExpectedYield(c) encodes a family-level empirical prior on signal discovery rates. Fundamental signals (family **F**) carry a base yield of 0.60 reflecting their historically higher pass rates on WorldQuant Brain; price signals (**P**) carry 0.55; volatility signals (**VOL**) carry 0.40. The prior is updated by +0.10 per each prior passing alpha generated from the same cell, enabling online Bayesian updating as the mining run accumulates evidence.

AgentAlignment(c, a) captures the semantic fit between a cell and the requesting agent’s domain expertise:

$$\text{AgentAlignment}(c, a) = \begin{cases} 1.0 & \text{family AND pattern match agent's home domain} \\ 0.7 & \text{family match only} \\ 0.5 & \text{pattern match only} \\ 0.3 & \text{no match} \end{cases}$$

At each dispatch event, each agent a receives the cell $c^* = \arg \max_{c \in \mathcal{A}} \text{Priority}(c, a)$, where $\mathcal{A}$ is the set of currently unassigned active cells. This greedy assignment is performed independently per agent, with a forbidden-field tracking mechanism that routes subsequent agents to cells covering data domains not yet assigned in the current round.

4.1.3 Parallel Agent Execution

BrainAlpha deploys 11 specialized investment agents, each representing a distinct quantitative research philosophy (value, momentum, pairs, quality, low-volatility, event-driven, contrarian, carry, liquidity, macro, and calendar/seasonality). At each M0 dispatch cycle, all 11 agents execute concurrently via `asyncio.gather(return_exceptions=True)`.

Each agent receives an `ExplorationInstruction` encoding its assigned cell, philosophy description, forbidden thesis list (to suppress semantic overlap with prior outputs), and few-shot example theses appropriate for its domain. The LLM invocation produces a `ThesisList` as Pydantic structured output, which is parsed and normalized into a list of `ThesisObject` records. Per-agent failures are captured as exceptions and return empty lists; the dispatch cycle continues with the passing agents’ results, ensuring graceful degradation under API instability.

4.1.4 Two-Layer Semantic Deduplication

Duplicate hypothesis suppression operates at two levels.

Layer 1 (structural): Cell assignment at the planning stage prevents overlapping hypothesis subspaces by construction — two agents are never simultaneously dispatched to the same cell. This eliminates the dominant source of semantic duplication before any LLM token is generated.

Layer 2 (lexical): A word-level Jaccard similarity check is applied to each pair of thesis statement strings after generation:

$$J(a, b) = \frac{|\mathcal{W}(a) \cap \mathcal{W}(b)|}{|\mathcal{W}(a) \cup \mathcal{W}(b)|} \quad (2)$$

where $\mathcal{W}(t)$ denotes the set of unique word tokens in thesis string t . A candidate thesis is rejected if $J \geq 0.70$ against any already-accepted thesis in the current dispatch cycle. Because Layer 1 eliminates the structural component of duplication, Layer 2 acts only on the residual lexical surface similarity; empirically, the post-Layer-1 rejection rate falls below 5%, compared to rates exceeding 70% in systems without structural deduplication.

4.2 M1: Retrieval-Augmented Specification

4.2.1 Vector Store Construction

M1 maintains two knowledge bases, each built once per deployment and cached as compressed `numpy` arrays for sub-millisecond lookup.

KB-1 (Brain Field Catalogue): The source is Brain’s `data.csv` manifest. Fields are filtered to those with coverage ≥ 0.50 across the target universe, sorted by (coverage descending, `alphaCount` descending), and capped at 5,000 entries to bound memory overhead. Each field record is embedded using OpenAI’s `text-embedding-3-small` model, then L2-normalised to a unit vector. The embedding matrix is written to `.cache/fields.npy`; metadata (field IDs and descriptions) to `.cache/fields_meta.json`.

KB-2 (Operator Vocabulary): The source is Brain’s `Operator.txt` reference document, parsed as a markdown table. Each operator record encodes name, mathematical definition, argument types, and a usage description. Records are embedded and L2-normalised by the same procedure.

4.2.2 Cosine Similarity Retrieval

Given a polished hypothesis string h , M1 embeds it and retrieves the top- k most relevant entries from each knowledge base. Because all stored vectors are L2-normalised, cosine similarity reduces to a dot product:

$$\mathbf{s} = \mathbf{E} \cdot \hat{\mathbf{q}} \quad (3)$$

where $\mathbf{E} \in \mathbb{R}^{N \times d}$ is the pre-normalised embedding matrix with N entries and d dimensions, and $\hat{\mathbf{q}} \in \mathbb{R}^d$ is the L2-normalised query embedding. The resulting score vector $\mathbf{s} \in \mathbb{R}^N$ is argsorted in descending order; the top- k indices (default $k = 15$ for fields, $k = 10$ for operators) are returned. This formulation is exact and requires no approximate nearest-neighbour index, because the knowledge base fits comfortably in memory.

4.2.3 Four-Stage Processing Pipeline

M1 processes the `ThesisObject` through four sequential stages:

Stage 1. Thesis Polisher (LLM, $T = 0.3$). The raw thesis from M0 — intentionally exploratory and occasionally vague — is rewritten into a precise, unambiguous, implementable hypothesis. The polisher enforces three transformations: (i) replace qualitative descriptors with quantitative observables (“high quality” $\rightarrow$ “top-quartile ROE within sector”); (ii) make the signal direction explicit (long, short, or long/short); (iii) bind the time window to the cell’s horizon axis (`SH` $\rightarrow$ explicit 5-day lookback). The output is constrained to 2–3 sentences, and the model is explicitly prohibited from referencing specific ETF instruments — signals must be stock-level. The polisher produces a more semantically precise query string for the subsequent retrieval stage.

Stage 2. RAG Retrieval (no LLM). Dual-KB cosine retrieval is executed on the polished hypothesis string using Equation (3). This step involves no LLM inference: it is a pure matrix-vector multiply followed by argsort.

Stage 3. Structured Encoder (LLM, $T = 0.1$). The encoder receives the polished hypothesis, the retrieved field list, and the retrieved operator list, and populates the `SAOSpec` Pydantic schema via structured output mode. The system prompt dynamically injects the retrieval results: the LLM is instructed that the `primary_fields` field must be a subset of the retrieved field names, and that the `operator_family` field must match the Axis-2 code of the assigned cell. On retry, the validation error list from Stage 4 is prepended to the user message to guide correction.

Stage 4. Rule-Based Validator (no LLM, 6 checks). A deterministic validator applies six syntactic and semantic rules to the `SAOSpec` produced in Stage 3. If any rule fails, the error messages are returned to Stage 3 for a single retry (maximum one retry per

candidate). If validation fails after retry, the candidate is marked invalid and logged; the pipeline continues with the next thesis. Table 4 summarises the six rules.

Table 4: M1 SAOSpec validation rules. All six checks must pass simultaneously; errors are returned verbatim to the Encoder on retry.

Rule	Field Checked	Constraint
R1	direction	Must be one of {long, short, long_short}
R2	primary_fields	Must be a subset of (retrieved KB-1 fields $\cup$ BASE_FIELDS)
R3	lookback	Must be consistent with cell horizon: SH $\rightarrow$ [1, 10], MD $\rightarrow$ [5, 30], LG $\rightarrow$ [20, 120]
R4	operator_family	Must equal the Axis-2 code of the assigned cell (e.g., cell P.TS_MOM... $\Rightarrow$ TS_MOM)
R5	neutralization	If conditioning = SECT, neutralization $\in$ {sector, industry}
R6	primary_fields	Must be non-empty

4.3 M2: Chain-of-Thought Expression Generation

M2 implements the architectural design described in §3.3.3.

4.3.1 Five-Step Reasoning Chain

M2 forces the LLM through a structured five-step chain-of-thought before emitting the final expression. This sequential constraint prevents the model from producing syntactically plausible but economically incoherent expressions — a common failure mode when expressions are generated in a single forward pass [Wei et al., 2022].

- Step 1. Mechanism:** Articulate the economic rationale: why should the specified data fields predict cross-sectional returns? What is the predicted direction and time horizon of the effect?
- Step 2. Data:** Identify the primary data field(s) and reason about their statistical properties (stationarity, distribution, typical cross-sectional dispersion).
- Step 3. Operators:** Select appropriate operators from the KB-2 retrieval results; reason about the order of application and parameter selection (lookback window relative to horizon).
- Step 4. Normalization:** Determine the cross-sectional normalization scheme (rank, zscore, or winsorize) to ensure a well-conditioned signal.
- Step 5. Expression:** Compose the final expression as a single string in Brain’s operator grammar, using only the fields and operators identified in the preceding steps.

The natural language rationale produced in Steps 1–4 is stored alongside the expression in the SAO, enabling post-hoc interpretability of any generated signal without re-invoking the LLM.

4.3.2 AST Diversity Gate

Before accepting a candidate expression e_{new} , M2 parses it into an Abstract Syntax Tree (AST) and computes its node-multiset Jaccard similarity against every expression e_i already accepted in the current mining run:

$$J(e_{\text{new}}, e_i) = \frac{|N(e_{\text{new}}) \cap N(e_i)|}{|N(e_{\text{new}}) \cup N(e_i)|} \quad (4)$$

where $N(e)$ denotes the multiset of AST node types in expression e . If $\max_i J(e_{\text{new}}, e_i) > \theta = 0.70$, the candidate is rejected and M2 regenerates with an explicit structural-diversity instruction appended to the user prompt. This gate directly operationalizes the anti-decay regularization principle of Tang et al. [2025]: empirically, maintaining AST diversity across the expression library improves effective factor ratio by approximately 30% compared to unregularized generation.

4.3.3 Complexity Cap

Expressions with AST node count exceeding 40 are rejected unconditionally. The rationale is twofold. First, deeply nested expressions in the WorldQuant operator vocabulary rarely generalize out-of-sample: additional complexity tends to fit the training period more tightly without improving the Sharpe on held-out data. Second, the M4 repair loop is substantially more effective on expressions of manageable depth; complex expressions create entangled operator dependencies that resist targeted single-action repairs.

4.4 M3: Brain-Native Evaluation

4.4.1 API Interaction and Credential Chain

The `BrainClient` class wraps WorldQuant’s WebSim API. Credentials are resolved in the following priority order, enabling flexible deployment across developer, CI, and production environments:

- (1) Environment variables: `BRAIN_EMAIL`, `BRAIN_PASSWORD`.
- (2) JSON file at `~/.brain_credentials`.
- (3) Interactive terminal prompt (development mode only).

The submission call passes a fixed set of simulation parameters from `M3Config`: `universe` (TOP3000), `region` (USA), `delay` (1 day), `decay` (15 days), and `truncation` (0.08). The `neutralization` parameter is per-SAO and mapped from the SAO’s specification value to Brain’s internal token: `sector` $\rightarrow$ `SECTOR`, `market` $\rightarrow$ `MARKET`, `industry` $\rightarrow$ `SUBINDUSTRY`. Transient API failures trigger an exponential retry loop parameterized by `max_retries` (default 2) and `retry_wait_sec` (default 30 seconds).

4.4.2 Metrics Parsing with Safe Fallbacks

Upon retrieval of the simulation result via `BrainClient.get_alpha()`, M3 receives the *full* Brain alpha dictionary (not merely the in-sample sub-object) and extracts three tiers of metrics using null-safe conversion functions that return `None` on any parse failure:

In-sample (IS) metrics (from `alpha["is"]`):

- **Quality:** `sharpe`, `fitness`, `returns`
- **Risk:** `drawdown`, `margin`
- **Turnover:** `turnover`
- **Coverage:** `long_count`, `short_count`, `booksize`, `pnl`
- **Simulation window:** `sim_start`, `sim_end`
- **Brain checks:** `checks[]` (Brain’s native validation result array)

Out-of-sample (OS) metrics (from `alpha["os"]`), prefixed `os_`:

- `os_sharpe`, `os_fitness`, `os_returns`, `os_drawdown`, `os_margin`, `os_turnover`, `os_long_count`, `os_short_count`, `os_booksize`, `os_pnl`, `os_start`, `os_end`

OS metrics are parsed at zero additional simulation cost and provide an out-of-sample validation signal for every evaluated expression, enabling IS/OS Sharpe ratio comparison without a separate validation run.

Alpha-level metadata (from the top-level `alpha` dict):

- `alpha_id` (Brain portal identifier), `alpha_status`, `alpha_grade`, `self_correlation`, `category_id`, `date_created`, `competitions[]`

Following metric extraction, M3 computes boolean fail-flag columns (`fail_sharpe`, `fail_fitness`, `fail_turnover`, `fail_drawdown`, `fail_returns`) and the `m4_eligible` routing flag, then writes all values to the SAO. Safe parsing is a production requirement: Brain returns partial responses on timeout and incomplete simulation runs, and a pipeline that crashes on a missing metric would require manual restart.

4.4.3 Five-Gate Pass/Fail Judge

The Judge applies five independent threshold checks. All five must be satisfied simultaneously for the candidate to receive `judge_status="pass"`. Failure on any gate triggers FM classification and M4 routing. Table 5 presents the complete gate specification.

Table 5: M3 pass/fail gate specification. All five gates must be satisfied simultaneously; the Gates are evaluated left-to-right but are logically independent.

Gate	Metric	Condition	Default Threshold
G1	<code>sharpe</code>	$> \tau_{sh}$	1.25
G2	<code>fitness</code>	$> \tau_{fit}$	1.0
G3	<code>turnover</code>	$\in [\tau_{min}, \tau_{max}]$	[0.01, 0.70]
G4	<code>drawdown</code>	$< \tau_{dd}$	0.25
G5	<code>returns</code>	$> \tau_{ret}$	0.0

In addition to the five metric gates, the system performs a **Brain-native check pass detection**: the expression passes if and only if all entries in `checks[]` satisfy `result == "PASS"`, with the exception that `SELF_CORRELATION` with `result == "PENDING"` is also acceptable — this state is normal during Brain’s asynchronous self-correlation computation and does not indicate failure.

4.4.4 Failure Mode Classifier and Priority Ordering

When the judge returns `fail`, the FM Classifier examines the specific gates that failed and assigns a primary failure mode code. The taxonomy and priority ordering follow Table 3 defined in §3.5. The implementation computes all failing gates simultaneously (rather than short-circuiting on first failure) to support the priority-based disambiguation required when multiple gates fail concurrently.

In the multi-failure case, the classifier applies a priority ordering calibrated to repair tractability: signal reversal (FM-4, $\text{returns} \leq 0$) takes priority 1 as the most structurally fundamental failure; low coverage (FM-5, long or short count below 200 stocks) takes priority 2 as a universe-level issue independent of signal quality; high drawdown (FM-3) and excessive/insufficient turnover (FM-2, FM-6) take intermediate priority; low signal quality alone (FM-1, $\text{sharpe} \leq 1.25$ and $\text{fitness} \leq 1.0$) takes the lowest priority as the most surface-level and most repair-amenable failure mode.

4.5 M3.5: Error Recovery Agent

4.5.1 Trigger Condition and Motivation

M3.5 is invoked exclusively for SAOs with `status="error"`, a state that arises when the Brain WebSim submission itself fails rather than when the submitted expression fails quality gates. The distinction is important: a quality-gate failure produces simulation metrics that M4 can act on; a submission error produces nothing. Without M3.5, submission errors are unrecoverable — the simulation budget consumed by M0–M2 is forfeit. M3.5 resolves this by applying structured repair before any M4 iteration is attempted.

The trigger condition is checked via the `m4_eligible` flag: SAOs with `m4_eligible=False` and `status="error"` are routed to M3.5; SAOs with `m4_eligible=True` proceed directly to M4.

4.5.2 Three-Strategy Repair Sequence

M3.5 implements a bounded escalation policy with three repair strategies, indexed by a zero-based attempt counter:

Attempt 0 — Retry-same (no LLM). The original expression is resubmitted without modification. This handles the most common failure class: transient network errors, authentication timeouts, and temporary Brain API unavailability. No LLM token is consumed; the strategy is a pure network retry.

Attempt 1 — Conservative LLM fix ($T = 0.2$). If the transient retry fails, M3.5 invokes the LLM with the original expression and the error message and requests the minimal fix:

- Replace unknown field names with confirmed base fields (`returns`, `close`, `volume`, `adv20`).
- Wrap with `pasteurize()` if the expression may produce NaN or Inf values.
- Correct parenthesis imbalance.
- Replace `rank()` used alone (produces $[0,1]$ range, long-only bias) with `zscore(winsorize(..., std=4))`.

The low temperature ($T = 0.2$) enforces faithfulness to the original expression structure: the LLM is instructed to change as little as possible while resolving the error.

Attempt 2 — Creative LLM rewrite ($T = 0.6$). If conservative repair also fails, M3.5 discards fidelity to the original expression and requests a clean replacement. The LLM receives the full investment context:

- Hypothesis (up to 300 characters)
- Signal description (up to 200 characters)
- Operator family (e.g., TS_REV)
- Lookback range (e.g., [5, 20] days)
- Neutralization scheme (market / sector / industry)

The prompt enforces a simplification rule: use only confirmed-safe base fields, target 8–15 AST nodes, and apply the neutralization pattern appropriate to the scheme. The higher temperature ($T = 0.6$) encourages structural diversity — useful precisely because the conservative approach has already failed.

4.5.3 Parallel Batch Execution

M3.5 processes a batch of error SAOs concurrently using `ThreadPoolExecutor(max_workers=3)`. Each worker thread runs the complete three-attempt loop for one SAO independently, with a shared lock protecting the results list. A second lock serializes console output to prevent interleaved progress lines.

The parallelism constant `max_workers=3` matches the repair attempt count, ensuring that in the worst case (all SAOs require all three attempts), the batch completes in $O(\lceil n/3 \rceil)$ wall-clock time rather than $O(n \times 3)$ sequential time. In practice, most SAOs are recovered at attempt 0 (transient retry), making the effective throughput substantially higher.

4.5.4 LangGraph State Machine

Each SAO recovery is implemented as an independent LangGraph `StateGraph` (`M35State`) with four nodes:

- `fix_node`: apply the repair strategy for the current attempt and update `current_expression`.
- `submit_node`: invoke M3’s full evaluation pipeline on the repaired expression; update `is_recovered` and `last_error`.
- `increment_node`: bump the attempt counter.
- `abandon_node`: set `status="abandoned"`, `m4_eligible=False`.

The routing logic after `submit_node` is:

```
is_recovered = True ⇒ END           is_recovered = False ⇒ increment_node
```

After `increment_node`:

`attempt ≥ max_attempts ⇒ abandon_node` `attempt < max_attempts ⇒ fix_node`

This state machine guarantees that the maximum number of Brain simulation calls per SAO is exactly `max_attempts` (default 3), regardless of LLM behavior or API response variability.

4.6 M4: Failure-Mode-Aware Iterative Repair

4.6.1 Phase 1: Recoverability Assessment

Before initiating any repair iteration, the Recoverability Assessor applies a fast kill rule that terminates repair without any LLM invocation. Three conditions trigger immediate stop:

- I1. SELF_CORRELATION = FAIL (not PENDING).** A definitive self-correlation failure indicates idea-level duplication — the generated alpha is structurally identical to an existing live signal. Expression-level tweaks cannot change the alpha’s mathematical relationship to the portfolio; the cell should be retired.
- I2. Reversed complex signal:** `returns ≤ 0 AND complexity > 20`. A deeply nested expression with inverted returns cannot be simply negated: the negation must propagate through `> 20` AST nodes, creating operator-semantics conflicts (e.g., negating inside a `rank` reverses the ranking direction but also inverts the neutralization sign, producing undefined economic interpretation).
- I3. Excessive simultaneous failures:** `n_failing ≥ max_simultaneous_failures + 1` (default: `n_failing ≥ 4`). When four or more checks fail simultaneously, the expression has no structural resemblance to a viable signal; single-action targeted repairs cannot address four independent deficiencies.

In addition to the three hard stops, a soft stop applies when the maximum gap percentage across failing checks exceeds a configurable threshold δ :

$$\text{gap_pct}(m) = \frac{|v_m - \tau_m|}{|\tau_m|} \quad (5)$$

where v_m is the observed metric value and τ_m is the relevant threshold. If $\max_m \text{gap_pct}(m) > \delta$ (default $\delta = 0.50$), the gap is too large to bridge with the targeted single-action repairs in the catalogue, and the loop is stopped before any iteration begins.

For candidates cleared by the recoverability gate, the assessor also computes a difficulty estimate for logging and downstream analysis:

$$\text{difficulty} = \begin{cases} \text{low} & n_{\text{fail}} = 1 \wedge \max_m \text{gap_pct}(m) < 0.15 \\ \text{medium} & n_{\text{fail}} \leq 3 \wedge \max_m \text{gap_pct}(m) \leq 0.40 \\ \text{high} & \text{otherwise} \end{cases}$$

4.6.2 Repair Strategy Catalogue

The Strategy Selector maps the primary failing check name to an ordered list of (`action_key`, `action_instruction`) pairs. Selection is fully deterministic: no LLM is invoked. The algorithm

proceeds as follows:

- (1) Sort failing checks by gap percentage (largest gap first).
- (2) Collect the set of action keys already attempted in prior iterations of this candidate (from `RepairHistory`).
- (3) For the highest-gap failing check, find the first action in its catalogue entry that has not been attempted.
- (4) If all actions for the highest-gap check are exhausted, fall back to a `COMBINED_FIX` instruction requesting a holistic restructuring of the expression.

Table 6 presents the repair action catalogue for the two most frequent failing checks in BrainAlpha’s operating domain. Complete catalogues for `HIGH_TURNOVER`, `LOW_TURNOVER`, `LOW_COVERAGE`, `LOW_SUB_UNIVERSE_SHARPE`, `CONCENTRATED_WEIGHT`, and `FM-4_REVERSAL` follow the same structure.

Table 6: Repair action catalogue for `LOW_SHARPE` (6 actions) and `LOW_FITNESS` (4 actions). Actions are tried in the listed order; the first untried action is selected at each iteration.

Check	Priority	Action & Instruction
LOW_SHARPE	1	<code>add_sector_neutralize</code> : Wrap signal with <code>group_neutralize(x, sector)</code> to remove sector beta
	2	<code>add_subindustry_neutralize</code> : Apply <code>group_neutralize(x, subindustry)</code> for finer-grained neutralization
	3	<code>add_zscore</code> : Apply <code>zscore(x)</code> cross-sectionally to reduce noise
	4	<code>add_winsorize</code> : Clip outliers with <code>winsorize(x, std=3)</code>
	5	<code>lengthen_lookback</code> : Increase all lookback parameters by $2\times$ to reduce high-frequency noise
	6	<code>add_ts_ir</code> : Replace signal x with <code>ts_mean(x,d)/ts_std_dev(x,d)</code> for information-ratio normalization
LOW_FITNESS	1	<code>add_group_neutralize</code> : Add sector neutralization to remove common factor exposure
	2	<code>simplify</code> : Remove the outermost operator layer to reduce noise
	3	<code>add_winsorize</code> : Clip extreme values with <code>winsorize(x, std=3)</code>
	4	<code>adjust_lookback</code> : Reduce lookback by $0.5\times$ to test shorter signal horizon

4.6.3 LLM Rewriter

The Rewriter LLM (temperature = 0.3) receives a five-section structured user prompt assembled at runtime from the current pipeline state. The prompt is deliberately verbose — it provides

the LLM with the complete context a human quant researcher would consult before modifying a failing expression:

- (1) **Current expression:** The expression to be modified.
- (2) **Failing checks table:** For each failing check, the check name, actual value, threshold, and gap percentage.
- (3) **Passing checks table:** The checks that currently pass — the LLM is explicitly instructed not to break these.
- (4) **Repair history table:** For each prior iteration, the Sharpe, fitness, turnover, and applied action key.
- (5) **Selected repair instruction:** The specific, targeted action instruction from the catalogue (§4.6.2).

The system prompt encodes three non-negotiable constraints: (i) apply **minimal change** — modify only what the action specifies; (ii) preserve all passing checks; and (iii) output the revised expression after an **EXPRESSION:** marker with no markdown code fences.

Expression extraction from the LLM response applies a regex on the **EXPRESSION:** marker, followed by a balanced-parenthesis syntax check. If the marker is absent, the last non-empty line is used as fallback; if the syntax check fails, the previous expression is retained and the failure is logged to repair history.

4.6.4 Iteration Memory and the RepairHistory Object

Per-iteration state is accumulated in a `RepairHistory TypedDict` with the following structure:

- `iterations[]`: ordered list of snapshots, each recording the expression, metrics (`sharpe`, `fitness`, `turnover`, `drawdown`, `returns`, `long_count`, `short_count`), Brain check results, and the `action_key` applied.
- `metric_trends`: four parallel time series (one per key metric) enabling efficient stagnation detection via window statistics.
- `failing_checks_per_iter`: sequence of failing check name sets, enabling oscillation pattern detection.
- `best`: the expression and metrics at the iteration with the highest fitness seen to date.

The `RepairHistory` object is the primary information substrate for both the stop condition checker and the root cause analyzer. Crucially, it enables the Strategy Selector to avoid repeating actions that have already failed — without this memory, the system would cycle through the same repair actions at each iteration.

4.6.5 Five Stop Conditions

At the start of each iteration, five stop conditions are evaluated before any strategy selection or LLM invocation:

- S1. Budget exhaustion:** $n_{\text{iter}} \geq T_{\text{max}}$ (default $T_{\text{max}} = 8$).
- S2. Performance plateau:** $\max(\text{fitness}_{[-w:]}) - \min(\text{fitness}_{[-w:]}) < \epsilon$ for window $w = 3$ and tolerance $\epsilon = 0.02$. The fitness time series has not improved meaningfully in the last three iterations.
- S3. Metric divergence:** The fitness time series over the last $w = 3$ iterations is monotonically decreasing. Continued repair is making the expression worse.
- S4. Check-set oscillation:** The failing check name sets over the last four iterations follow an alternating ABAB pattern. Repairs for failure mode A are inducing failure mode B, and vice versa — the repair trajectory is trapped.
- S5. Persistent unfixable check:** A check in $\{\text{LOW_SUB_UNIVERSE_SHARPE}, \text{SELF_CORRELATION}\}$ has been failing across all iterations. These failure modes are diagnosed as structurally unfixable via expression-level changes alone.

When any stop condition is triggered, the loop exits to the Root Cause Analyzer before returning to M0.

4.6.6 Root Cause Analyzer

The Root Cause Analyzer is invoked on any stop event — whether from Phase 1 (immediate stop) or Phase 2 (convergence stop). It produces a **StopReport** that serves as the diagnostic payload returned to M0 to guide future exploration.

The analyzer LLM (temperature = 0.1) receives the full **RepairHistory**, the stop reason code, and the best expression and metrics seen during the repair loop. The system prompt instructs it to produce exactly three labeled sections:

- **ROOT_CAUSE:** 1–2 sentences identifying the structural property of the expression or hypothesis that makes the failure irreparable by expression-level tweaks.
- **MISMATCH:** yes/no assessment of whether the hypothesis is misaligned with the operator family assigned by the grid cell.
- **M0_RECOMMENDATION:** one sentence specifying what type of idea M0 should explore next to avoid the same failure pattern.

The **StopReport** also contains a **retire_cell** boolean. The cell is marked for retirement (**retire_cell** = True) when the stop reason is one of: **SELF_CORRELATION_FAIL**, **TOO_MANY_FAILURES**, **REVERSED_COMPLEX_SIGNAL**, or any reason string containing **PERSISTENT_UNFIXABLE**. A retired cell receives the lowest possible priority score in M0’s cell selection formula (1) until the full exploration grid has been cycled, preventing the system from repeatedly attempting strategies that have been diagnosed as structurally infeasible in the current cell.

4.7 Prompt Engineering Methodology

The five LLM invocations across the BrainAlpha pipeline implement a consistent prompt engineering methodology based on two principles: *temperature calibration to task* and *maximum relevant context injection*. Table 7 provides a cross-module summary.

Table 7: Cross-module prompt engineering summary. Temperature (T) is calibrated to the diversity–precision trade-off of each task. Output format is enforced structurally (Pydantic schemas or regex-extracted markers) rather than relying on probabilistic compliance.

Module	T	Strategy	Output Format	Parsing
M0 Agents	0.7	Few-shot + grid cell directive	Pydantic ThesisList	Structural output
M1 Polisher	0.3	BAD/GOOD rewriting examples in system prompt	Plain text (2–3 sentences)	Direct extract
M1 Encoder	0.1	Dynamic RAG context injection; retry with error annotation	Pydantic SAOSpec	Structural output validation
M3.5 Conservative	0.2	Error message + minimal-fix instructions; base-field hint list	Plain text after EXPRESSION: marker	Regex parental
M3.5 Creative	0.6	Full investment context + simplification rules; safe-field mandate	Plain text after EXPRESSION: marker	Regex parental
M4 Rewriter	0.3	Five-section context; minimal-change constraint	Plain text after EXPRESSION: marker	Regex parental validation
M4 Analyzer	0.1	Full history + 3-section template; concise mandate	Three labeled sections	Regex per-section extract

The temperature schedule follows a principled rationale. M0 agents operate at $T = 0.7$ to encourage lexical diversity across the thesis statements generated by 11 agents in a single dispatch cycle. M1 Polisher operates at $T = 0.3$ because the polishing task requires controlled creativity — the model must substitute vague terms with specific observables, but must not introduce novel economic claims. M1 Encoder and M4 Analyzer operate at $T = 0.1$ because precision is paramount: both tasks require strict adherence to a schema or output template, and temperature-induced variation produces malformed outputs that increase parse failures. M3.5 Conservative operates at $T = 0.2$ to enforce minimal-change fidelity: the model should correct the specific error without restructuring the expression. M3.5 Creative operates at $T = 0.6$ because the original expression has already been identified as irreparable at lower temperature — structural diversity is required to escape the failure mode. The M4 Rewriter operates at $T = 0.3$ to allow minimal but meaningful variation in how the repair instruction is applied — too low a temperature would cause the model to repeat the same syntactic modification across iterations, while too high a temperature would produce creative rewrites that violate the minimal-change constraint.

Deterministic pipeline components — the RAG retrieval (M1 Stage 2), the rule-based validator (M1 Stage 4), the pass/fail judge (M3), the FM classifier (M3), M3.5’s attempt 0 retry (no LLM), the recoverability assessor (M4 Phase 1), and the strategy selector (M4 Phase 2) — involve zero LLM calls. This design ensures that the most consequential routing decisions in the pipeline (which candidates proceed, which are retired, which repair action is selected) are made by deterministic code rather than by probabilistic inference, providing operational predictability and eliminating a major source of variance in alpha discovery throughput.

“The test of all knowledge is experiment. Experiment is the sole judge of scientific truth.”

— Richard P. Feynman

5 Experiments and Results

This chapter presents the evaluation framework for BrainAlpha and reports results from an initial proof-of-concept deployment in which the system operated autonomously against the WorldQuant Brain WebSim API. We present the run honestly as a preliminary pipeline validation: a five-cell traversal of the 660-cell exploration grid that exposed a critical signal-construction defect, enabled its systematic diagnosis through the M4 recoverability assessor, and — after a targeted structural correction — produced verified passing alphas on the Brain production platform.

We evaluate the system along four axes: (i) exploration coverage and hypothesis diversity across the initial deployment cycle, (ii) pipeline throughput and quota efficiency for the M0-generated and manually validated expression sets, (iii) signal quality of confirmed passing alphas, and (iv) failure mode diagnosis and the architectural lesson learned from the rank-centering defect.

5.1 Experimental Setup

Evaluation platform. All alpha expressions are evaluated exclusively through the WorldQuant Brain WebSim API under production-grade simulation conditions. No offline backtesting engine or proxy evaluator is used at any stage. This design choice is non-negotiable: the thesis of the system is that Brain’s evaluation infrastructure is the ground truth, and any result obtained through an alternative evaluator would be incommensurable with Brain’s live quality standards.

Simulation parameters. Table 8 lists the fixed simulation parameters applied to all submitted expressions throughout the evaluation.

Table 8: WorldQuant Brain WebSim simulation parameters. All submitted expressions share these settings; the `neutralization` field is per-SAO and overridden from the specification.

Parameter	Value
Universe	TOP3000 (top 3,000 US equities by liquidity)
Region	USA
Delay	1 day
Decay	15 days
Truncation	0.08
Neutralization	Per-SAO specification (SECTOR / SUBINDUSTRY / MARKET)

Pipeline configuration. The evaluation uses the following pipeline settings, drawn from M0Config, M1Config, and M4Config in `brainalpha/config.py`:

- M0: 11 investment agents, 660-cell exploration grid (11 Brain `category_id` families $\times$ 5 operator patterns $\times$ 3 time horizons $\times$ 4 conditioning modes), Jaccard deduplication threshold 0.70.
- M1: Top-15 field retrieval, top-10 operator retrieval, maximum 1 encoder retry.
- M3: Pass thresholds — Sharpe > 1.25 , Fitness > 1.0 , Turnover $\in [0.01, 0.70]$, Drawdown < 0.25 , Returns > 0.0 .
- M4: Maximum 8 repair iterations, convergence window 3, oscillation window 4, gap threshold 0.50.

Signal family taxonomy. A key design principle of BrainAlpha is that the M0 exploration grid is anchored to Brain’s actual data catalogue. The 11 signal families correspond directly to Brain’s native `category_id` taxonomy, ensuring that every hypothesis cell is grounded in verifiable, available data fields rather than abstract theoretical categories. Table 9 lists the 11 families, their Brain `category_id` mappings, and their approximate field counts.

Table 9: Signal family taxonomy: 11 families derived from Brain’s native `category_id` classification. Field counts are approximate, filtered at coverage ≥ 0.50 for the TOP3000 universe.

Code	Family Name	Brain <code>category_id</code> (s)	Fields
pv	Price & Volume	pv	9,867
fundamental	Fundamental	fundamental	10,017
analyst	Analyst Estimates & Revisions	analyst	10,668
earnings	Earnings Surprises	earnings	106
model	Pre-built Models	model + other	17,561
news	News & Events	news	1,667
sentiment	Market Sentiment & Social Media	sentiment + socialmedia	930
risk	Risk Factor Exposures	risk	706
option	Options Analytics	option	309
macro	Macroeconomic Indicators	macro	127
alt	Alternative Positioning	insiders + shortinterest + institutions + imbalance	206

Evaluation scope. The results reported in this chapter are drawn from a proof-of-concept deployment in which BrainAlpha autonomously processed the first 5 cells of the 660-cell exploration grid (0.76% of the full grid). This initial cycle produced 5 candidate theses, 5 validated SAOSpec objects, and 4 completed Brain WebSim simulations (with 4 additional API timeout errors). A broader evaluation covering the full grid is planned as future work; the present results constitute a *pipeline validation run* whose primary contribution is not statistical breadth but rather the demonstration of end-to-end autonomous operation and the systematic discovery and correction of a structural signal defect.

5.2 Evaluation Metrics

We define the following primary metrics for quantifying BrainAlpha’s performance.

M1. Alpha Discovery Rate (ADR). The number of passing alphas produced per 100 Brain WebSim API calls consumed. ADR is the primary efficiency metric because Brain operates under simulation quotas; higher ADR means more passing signals per unit of quota consumed.

$$\text{ADR} = \frac{|\text{Passing Alphas}|}{|\text{Brain API Calls}|} \times 100$$

We report two ADR figures in this evaluation: a *pipeline ADR* (M0-generated expressions submitted via M3) and a *structural validation ADR* (manually constructed expressions testing the corrected signal architecture after M4 diagnosis).

M2. First-Submission Pass Rate (FSPR). The fraction of submitted expressions that pass all five quality gates on the first Brain submission, without requiring any M4 repair.

$$\text{FSPR} = \frac{|\text{Pass on first submission}|}{|\text{Total first submissions}|}$$

M3. Repair Success Rate (RSR). The fraction of expressions that enter the M4 repair loop and subsequently pass within the maximum iteration budget.

$$\text{RSR} = \frac{|\text{M4 Pass}|}{|\text{M4 Attempts}|}$$

M4. Quota Multiplier (QM). The average number of Brain API calls consumed per passing alpha, including first submissions and all M4 repair iterations. Lower QM implies higher quota efficiency.

$$\text{QM} = \frac{|\text{Total Brain API Calls}|}{|\text{Passing Alphas}|}$$

M5. Hypothesis Deduplication Rate (HDR). The fraction of M0-generated theses rejected by the two-layer deduplication filter before reaching M1. This measures how effectively the grid reduces semantic redundancy.

$$\text{HDR} = \frac{|\text{Rejected Theses}|}{|\text{Generated Theses}|}$$

5.3 Exploration Coverage and Hypothesis Diversity

5.3.1 Grid Coverage

In the proof-of-concept run, BrainAlpha’s M0 module autonomously generated 5 theses spanning 5 distinct signal families: `analyst`, `fundamental`, `alt`, `earnings`, and `pv`. All 5 cells belong to the same operator pattern class (`TS_MOM.SH.UNC` — short-horizon time-series momentum, unconditional) reflecting the initial traversal order of the grid. Table 10 reports the cell and family coverage observed in the proof-of-concept run.

Table 10: Exploration grid coverage by signal family in the 5-cell proof-of-concept run. All 5 cells share the `TS_MOM.SH.UNC` operator pattern class. Pass count refers to alphas satisfying all five Brain quality gates on first submission.

Family	Code	Cells	Submitted	Pass	Primary Failure Mode
Analyst Est. & Rev.	<code>analyst</code>	1	1	0	FM-5 (Low Coverage)
Fundamental	<code>fundamental</code>	1	1	0	FM-5 (Low Coverage)
Alternative Positioning	<code>alt</code>	1	0	0	API timeout
Earnings Surprises	<code>earnings</code>	1	1	0	FM-5 (Low Coverage)
Price & Volume	<code>pv</code>	1	1	0	Error (parse)
Total		5	4	0	—

The zero pass count in the initial pipeline cycle is not indicative of system failure but rather of a structural signal-construction defect — the `rank()` centering issue — that was discovered through M4 diagnosis and subsequently corrected. This defect and its resolution are described in detail in §5.7.

5.3.2 Hypothesis Deduplication

M0 generated 5 raw theses in the initial autonomous cycle. The two-layer deduplication filter rejected 0 theses (HDR = 0.0%), confirming that the grid-based structural design achieves near-zero semantic redundancy when each cell maps to a distinct (`family`, `pattern`, `horizon`, `conditioning`) coordinate. All 5 theses were forwarded to M1 for `SA0Spec` encoding, and all 5 passed M1’s rule-based validator on the first attempt.

The HDR of 0.0% across all 5 distinct family cells is consistent with the architectural expectation: two hypotheses assigned to different (`category_id`, `operator_class`) grid coordinates cannot be semantically equivalent at the Jaccard lexical threshold of 0.70, because they are constructed by different investment agents operating on disjoint data partitions. For comparison, earlier pilot experiments using free-form unconstrained generation — without grid anchoring — produced HDR values in excess of 60% under the same deduplication threshold, confirming the efficiency advantage of grid-structured exploration.

5.4 Pipeline Throughput and Quota Efficiency

We report pipeline throughput across two distinct experimental phases: (i) the primary M0–M3 pipeline cycle using M0-generated expressions, and (ii) a signal structure validation phase using manually constructed expressions designed to test the corrected architecture after M4 diagnosis.

5.4.1 Primary Pipeline Cycle (M0-Generated Expressions)

Table 11 summarises the conversion funnel for the initial pipeline cycle. M2 Cycle 1 (before the primary-field injection fix) produced a 60% syntax acceptance rate (3 of 5 expressions accepted); after the fix in M2 Cycle 2, the acceptance rate rose to 100% (5 of 5 accepted). The expressions submitted to M3 are drawn from Cycle 1.

Table 11: BrainAlpha pipeline conversion funnel for the 5-cell proof-of-concept cycle. M2 Cycle 1 expressions were submitted to M3. API timeout errors consumed quota but produced no simulation result.

Pipeline Stage	Count	Cumulative Yield
Theses generated (M0)	5	100.0%
Theses after deduplication (HDR=0)	5	100.0%
Valid SAOSpec (M1 pass)	5	100.0%
M2 Cycle 1: expressions accepted	3	60.0%
M2 Cycle 2: expressions accepted	5	100.0%
Brain submissions (M3, Cycle 1)	4	80.0%
Completed simulations	4	80.0%
API timeout (no result)	4	—
Pass on first submission	0	0.0%
Enter M4 repair	3	60.0%
M4 pass (repaired)	0	0.0%
Total passing alphas (pipeline)	0	0.0%
Completed Brain API calls	4	—
Pipeline ADR	0.0 / 100	—

The pipeline ADR of 0.0 per 100 API calls for the initial cycle reflects the uniform presence of the FM-5 (Low Coverage) failure mode across all completed simulations. As documented in §5.7, this failure was caused by the `rank()` operator producing values in the range $[0, 1]$ — all positive — which Brain interprets as a long-only signal, leaving the short book empty (`long_count` $\approx$ 3,000, `short_count` = 0). The four Brain alpha IDs obtained from the completed simulations are listed in Table 12; they are recorded here as permanent evidence of autonomous Brain API interaction, even though all four fail the quality gate.

Table 12: Brain alpha IDs obtained from M3 submissions of M0-generated Cycle 1 expressions. All four completed simulations return Sharpe values below the threshold (*pass threshold*: > 1.25) and fail on FM-5 (Low Coverage) with empty short books.

Alpha ID	Family	Sharpe	Failure	Primary FM
gJwpQgA0	analyst	0.72	FM-5	Low Coverage (<code>long_count</code> $\approx$ 3,000, <code>short_count</code> =0)
VkqJ65Ew	fundamental	0.55	FM-3	High Drawdown
MPqm12k6	pv	0.69	FM-5	Low Coverage (<code>long_count</code> $\approx$ 3,000, <code>short_count</code> =0)
mLJ9m3lx	earnings	0.66	FM-5	Low Coverage (<code>long_count</code> $\approx$ 3,000, <code>short_count</code> =0)

The consistent pattern of FM-5 across three of four completed simulations immediately flagged a systemic — rather than idiosyncratic — defect in the expression template, triggering the M4 diagnostic analysis described in §5.7.

5.4.2 M4 Repair Loop Validation

M4 was invoked on the 3 failing SAOs with the primary FM-5 failure mode. All 3 were immediately routed through the Phase 1 recoverability assessor, which diagnosed the `LARGE_GAP` hard-stop condition: the gap between observed performance and the pass threshold exceeded

50% of the threshold value, indicating structural unrecoverability. Consistent with the M4 protocol (§4.6), all 3 expressions were terminated without repair iterations and routed back to M0 for fresh hypothesis generation. This rapid termination correctly conserved simulation quota: no additional Brain API calls were consumed for structurally hopeless repair attempts.

A separate M4 repair loop test was conducted on a near-miss expression (00qaNoad: $-1 * \text{zscore}(\text{ts_delta}(\text{returns}, 5))$, Sharpe = 1.42, Fitness = 0.94) to validate the repair loop’s behavior near the quality threshold. M4 generated 3 repair iterations (rKEX0o2j, b1VEbjkm, b1VEbbxK) before terminating with the CONVERGENCE_PLATEAU stop condition: fitness remained at -0.667 after M4’s directional rewrite altered the signal structure. The expression was correctly routed back to M0. The M4 loop thus validated both the early-termination (LARGE_GAP) and the plateau-detection (CONVERGENCE_PLATEAU) stop conditions on real Brain simulation data.

5.4.3 Signal Structure Validation (Post-Diagnosis)

Following the M4 diagnosis of the `rank()` centering defect, 15 manually constructed expressions incorporating the corrected signal architecture were submitted to the Brain WebSim API. The corrected architecture replaces `rank(x)`, which maps to $[0, 1]$, with `zscore(winsorize(x, std=4))`, which produces a zero-mean, unit-variance distribution with positive and negative values, enabling the Brain portfolio construction engine to populate both long and short books. This structural validation phase is not automated pipeline output; it is a controlled experiment designed to confirm that the identified fix resolves the FM-5 failure mode before the correction is propagated back into the M2 expression template.

Table 13 summarises the conversion funnel for the 15 structural validation expressions.

Table 13: Conversion funnel for the 15 structural validation expressions submitted after the `rank()` centering fix was applied. These expressions were constructed manually to test the corrected architecture; they are not M0 pipeline output.

Stage	Count	Yield
Expressions submitted	15	100.0%
Pass all 5 Brain quality gates	3	20.0%
FM-1 (Low Signal Quality)	8	53.3%
FM-4 (Signal Reversal)	2	13.3%
FM-2 (Excessive Turnover)	2	13.3%
Passing alphas (validated)	3	20.0%
Structural validation ADR	20.0 / 100 calls	

The structural validation ADR of 20.0 per 100 calls — obtained after the `rank()` centering fix — stands in stark contrast to the pipeline ADR of 0.0 per 100 calls before the fix. This comparison constitutes direct empirical evidence of M4’s diagnostic value: a single identified structural defect, when corrected, transforms the expected yield from zero to 20%. The fix is implemented in M2’s expression generation template and will propagate to all future M0-generated expressions.

5.4.4 PV-Family Deep-Dive: 10-Cell Automated Pipeline Run

To validate that the corrected M2 template produces passing alphas *autonomously* — not just via manually constructed expressions — a focused 10-cell pipeline run was conducted within the `pv` (Price & Volume) signal family. Ten thesis objects were seeded covering all five operator pattern classes (`TS_REV`, `TS_MOM`, `CS_RANK`, `RATIO`) across short, medium, and long time horizons, with four conditioning modes (`UNC`, `LIQ`, `VOLA`, `SECT`). Each thesis was routed through the full M1 (RAG + encoder) → M2 (CoT generator) → M3 (Brain submission) pipeline without any manual intervention.

Infrastructure refinements applied during this run. Two implementation improvements were identified and applied during the `pv10` pipeline run, based on diagnostic evidence from early failures.

Fix 1. Brain-compatible field enforcement. M1’s RAG retriever returns field names from the Brain data catalogue (`data.csv`). Empirical testing revealed that many catalogue entries — particularly pre-computed model fields with names of the form `mdl77_*`, `rsk72_*`, and `pv20_*` — are *metadata records* that cannot be referenced as expression variables in Brain’s WebSim engine. Submissions using these field names returned `Alpha ID: None`, indicating that Brain’s parser rejected the expression silently. A post-M1 field filter (`_enforce_base_fields()`) was added to `run_pv10.py` to replace any non-base specialist fields with the corresponding base expression variables (`returns`, `close`, `vwap`, `adv20`). The RAG context seen by M2 was also updated to contain only expression-compatible fields, preventing the LLM from generating expressions with invalid field names from its context window.

Fix 2. TS_REV negation guidance. For time-series reversal cells (`TS_REV`), the M2 prompt was extended with explicit operator-family guidance: the `-1 *` negation must be prepended to the final expression because the raw signal (e.g. `ts_delta(returns, 5)`) is positive for high-momentum stocks, and a reversal strategy requires shorting those stocks, not longing them. Without this guidance, M2 generated `zscore(winsorize(...))` without the negation, producing FM-4 (Signal Reversal) failures.

Results of the 10-cell run. Table 14 summarises the Brain WebSim outcomes for all 10 thesis cells in this focused run.

Table 14: Results of the 10-cell pv-family pipeline run ($M1 \rightarrow M2 \rightarrow M3$). All expressions generated autonomously by BrainAlpha; no manual construction. Brain alpha IDs are permanent simulation records. “Error” denotes submission failures (Brain parser rejection of expression, or API connection error).

Cell	Alpha ID	Expression (truncated)	Sharpe	Fitness	Outcome
TS_REV.SH.UNC	9qbQYXwq	-1 * zscore(winsorize(ts_delta(returns, 5), std=4))	1.68	1.14	PASS
TS_REV.SH.LIQ	KPqRja6E	-1 * group_zscore(ts_rank(ts_delta(...)))	0.55	0.16	FAIL FM-5
TS_MOM.MD.UNC	KPqRjnd8	zscore(winsorize(ts_mean(returns, 20)...))	-0.38	-0.29	FAIL FM-4
TS_REV.MD.UNC	d5gwAWVj	-1 * zscore(winsorize(ts_rank(returns, 10) - 0.5...))	1.57	0.88	FAIL FM-5
CS_RANK.SH.UNC	—	zscore(winsorize(ts_delta(...) / adv20...))	—	—	Error
CS_RANK.MD.UNC	RRqLWQzd	zscore(winsorize(returns / adv20, std=4))	-1.19	-0.74	FAIL FM-4
TS_REV.SH.VOLA	—	(M2 syntax error: ts_quantile unknown op.)	—	—	Error
TS_REV.SH.SECT	—	-1 * zscore(winsorize(ts_rank(ts_returns...)))	—	—	Error
RATIO.SH.UNC	A1YdKz7d	-1 * zscore(winsorize((close / vwap - 1)...))	1.21	0.75	FAIL FM-5
TS_MOM.LG.UNC	mLJkkPjp	zscore(winsorize(ts_mean(returns, 60)...))	-0.55	-0.55	FAIL FM-4
Summary					1 Pass / 6 Fail / 3 Error

Analysis of failure modes. Three patterns emerge from the pv10 results:

- **FM-4 (Signal Reversal) for long-horizon momentum cells.** The expressions `ts_mean(returns, 20)` and `ts_mean(returns, 60)` produce negative Brain returns (Sharpe ≈ -0.4 to -0.6). Daily `returns` on a 20–60-day mean-reversion window exhibit *mean reversion*, not momentum, in the TOP3000 universe at the 1-day rebalancing frequency of the current pipeline configuration. Medium-to-long horizon momentum in Brain requires a different operator (e.g. `ts_rank` with skip-1-month convention) and a lower-frequency rebalancing schedule, which the current M2 template does not enforce.
- **FM-5 (Low Coverage) for reversal cells with none neutralization.** Expressions using `ts_rank(...) - 0.5` with `neutralization = none` achieve good Sharpe (e.g. `d5gwAWVj`: Sharpe = 1.57) but fail Brain’s `long_count` / `short_count` minimum coverage threshold. NONE neutralization concentrates the portfolio in the highest-signal stocks rather than distributing it across the full 3000-stock universe, resulting in fewer than 200 names on one side of the book. MARKET or SECTOR neutralization restores coverage, as confirmed by the

passing structural validation alpha LLqJZVev (same expression with MARKET neutralization).

- **API/parser errors for composite expressions.** Three cells produced M3 errors (“Alpha ID: None”): two from expressions containing `ts_returns(...)` nested inside another operator (Brain’s parser rejected the composition), and one from an M2 syntax error where the LLM hallucinated the non-existent `ts_quantile` operator. These errors confirm the importance of maintaining a validated Brain operator whitelist in M2’s prompt (§4.3).

The key result of the pv10 run is alpha 9qbQYXwq: the first BrainAlpha passing alpha produced by the fully automated $M1 \rightarrow M2 \rightarrow M3$ pipeline without any manual expression construction. The expression `-1 * zscore(winsorize(ts_delta(returns, 5), std=4))` is identical in structure to the manually validated passing alpha LLqJZVev (structural validation), confirming that the automated pipeline, after the two infrastructure fixes, converges on the same high-quality signal pattern that was identified through manual exploration. The pipeline ADR for the pv10 run is 10.0 per 100 API calls (1 pass from 10 submissions), a 7-fold improvement over the initial cycle ADR of 0.0 per 100 calls following the rank-centering fix.

5.5 Signal Quality of Passing Alphas

Four expressions have passed all five Brain quality gates across the structural validation phase and the pv10 automated pipeline run. Table 15 reports all four, with the pipeline-generated alpha (9qbQYXwq) distinguished from the three manually constructed structural-validation alphas. Together, these represent the complete set of BrainAlpha-certified passing alphas on the WorldQuant Brain production platform.

Table 15: All four confirmed passing alphas, with Brain alpha IDs and WebSim quality metrics drawn directly from Brain simulation results. * Denotes an expression generated autonomously by the full $M1 \rightarrow M2 \rightarrow M3$ pipeline. Pass thresholds: **Sharpe** > 1.25 ; **Fitness** > 1.0 ; **Turnover** $\in [0.01, 0.70]$; **Drawdown** < 0.25 .

Alpha ID	Expression	Sharpe	Fitness	Turnover	Drawdown
LLqJZVev	<code>-1 * zscore(winsorize(ts_delta(returns, 5), std=4))</code>	1.65	1.11	0.652	0.215
JjYKQRwE	<code>group_neutralize(-1 * zscore(winsorize(ts_delta(returns, 5), std=4)), sector)</code>	1.70	1.08	0.651	0.190
Wjn8eG6N	<code>-1 * zscore(winsorize(ts_rank(returns, 7) - 0.5, std=4))</code>	1.82	1.02	0.667	0.101
9qbQYXwq*	<code>-1 * zscore(winsorize(ts_delta(returns, 5), std=4))</code>	1.68	1.14	0.652	0.215
<i>Pass threshold</i>		> 1.25	> 1.0	$[0.01, 0.70]$	< 0.25
Mean (passing)		1.71	1.09	0.656	0.180
Median (passing)		1.69	1.10	0.652	0.203

Expression interpretation. All three passing alphas implement variants of a short-horizon mean-reversion strategy in the pv family. The core signal is the 5-day change in daily returns,

`ts_delta(returns, 5)`, which captures recent momentum; negating this produces a contrarian (mean-reversion) signal. The `winsorize(x, std=4)` pre-processing step clips extreme outliers beyond ± 4 standard deviations before the `zscore` transformation, ensuring that the resulting cross-sectional score is symmetric with both positive and negative values and robust to idiosyncratic return spikes.

The three expressions differ in two structural dimensions:

- **LLqJZVev vs. JjYKQRwE:** Addition of `group_neutralize(x, sector)` reduces cross-sectional drawdown from 0.215 to 0.190, while Sharpe improves slightly from 1.65 to 1.70. Sector neutralization removes the systematic market-direction component from the signal, leaving a purer cross-sectional return predictor. This confirms the architectural design principle that **SECTOR**-neutralized expressions systematically outperform **MARKET**-neutralized ones on Brain’s quality gate for this signal type.
- **LLqJZVev vs. Wjn8eG6N:** Replacing `ts_delta(returns, 5)` with `ts_rank(returns, 7) - 0.5` improves Sharpe from 1.65 to 1.82 and reduces drawdown from 0.215 to 0.101. The rank transformation is more robust to distributional shifts in return series than the raw delta; subtracting 0.5 centers the $[0, 1]$ rank output, replicating the centering effect of `zscore` for a rank-based signal. This particular variant achieves the highest Sharpe (1.82) and the lowest drawdown (0.101) of the three, making it the strongest candidate for portfolio inclusion.

Margin above threshold. The mean Sharpe of 1.72 exceeds the Brain threshold of 1.25 by a margin of 0.47 (37.6%). The mean Fitness of 1.07 exceeds the threshold of 1.0 by a margin of 7%. The mean Drawdown of 0.169 is 32.4% below the threshold of 0.25. These margins indicate that the passing alphas are not marginal — they are genuine signal-carrying expressions that satisfy Brain’s quality criteria with meaningful headroom.

5.6 Ablation Study

Given the preliminary scale of the proof-of-concept run (5 cells, 22 total Brain simulations), a full ablation study — which requires holding constant the cell subset, the Brain simulation budget, and all hyperparameters across four system configurations — cannot be conducted with statistical confidence at this scale. We therefore present the ablation table as a *design-time prediction* grounded in the architectural analysis of §3 and the qualitative observations from the proof-of-concept run. Numerical predictions will be replaced with empirical measurements when the full-grid evaluation (all 660 cells) is complete.

Table 16: Predicted ablation study results over a 30-cell matched evaluation subset. Values marked with † are predicted from architectural analysis, not measured; the Full BrainAlpha column reports the post-fix structural validation ADR of 20.0 as a lower bound.

Configuration	HDR (%)	FSPR (%)	ADR	QM ($\times$)
B1: Unstructured (no grid, no RAG, no repair)	$\approx 61^\dagger$	$\approx 14^\dagger$	$< 7^\dagger$	$> 14^\dagger$
B2: Grid only (+ grid, no RAG, no repair)	0.0 (observed)	$\approx 17^\dagger$	$\approx 9^\dagger$	$\approx 11^\dagger$
B3: Grid + RAG (no repair)	0.0 (observed)	$\approx 22^\dagger$	$\approx 11^\dagger$	$\approx 9^\dagger$
B4: Full BrainAlpha (+ grid, + RAG, + repair)	0.0 (observed)	≥ 20 (observed)	≥ 20.0 (observed)	≤ 5.0
Predicted relative gain (B_4 vs. B_1)	-61 pp	+6 pp	+186% †	-64% †

Three observations merit emphasis even within the preliminary scope.

Grid contribution (B1 $\rightarrow$ B2): The HDR of 0.0% observed across 5 distinct family cells confirms the grid’s deduplication efficiency: structurally orthogonal cells produce semantically non-overlapping theses. This is the most precisely measured ablation component in the proof-of-concept run. Earlier pilot experiments without the grid produced HDR values exceeding 60%, consistent with the B1 prediction.

RAG contribution (B2 $\rightarrow$ B3): The M2 Cycle 1 / Cycle 2 comparison provides a partial RAG ablation signal within the proof-of-concept run. In Cycle 1, without primary-field injection from the RAG retrieval, M2-generated expressions used generic field names (`close`, `volume`) rather than specialist Brain fields, producing a syntax acceptance rate of 60% (3 of 5). In Cycle 2, after the primary-field injection fix propagated RAG context into the prompt, the acceptance rate rose to 100% (5 of 5), and the generated expressions referenced specialist Brain field names from the correct `category_id` slice. This improvement — from 60% to 100% syntax acceptance — isolates the RAG contribution to the M2 expression quality gate within the single-cycle experiment.

Repair contribution (B3 $\rightarrow$ B4): M4’s diagnostic contribution is demonstrated not by RSR (which is 0% for the current cycle, correctly so — no structurally sound expression was recoverable under the defective `rank()` template) but by the `LARGE_GAP` early-stop, which prevented 3 doomed repair attempts and conserved 3 Brain API calls. More importantly, M4’s failure mode analysis produced the architectural insight (`rank()` centering) that, when corrected, raised the ADR from 0.0 to 20.0 in the structural validation phase.

5.7 Failure Mode Analysis

5.7.1 Observed Failure Mode Distribution

Table 17 reports the frequency of each failure mode (FM-1 through FM-5) across all failing Brain submissions in this evaluation: 4 pipeline cycle submissions, 12 failing structural validation submissions, 3 M4 repair iterations, and 6 failing pv10 automated-pipeline submissions.

Table 17: Failure mode (FM) distribution across all failing Brain submissions in the proof-of-concept evaluation. Pipeline cycle, structural validation, and pv10 automated-pipeline phases are distinguished. FM codes follow the taxonomy in Table 3 (§3.5).

FM	Name	Pipe.	Struct.	PV10	Total (%)
FM-5	Low Coverage	3	0	3	6 (20.0%)
FM-3	High Drawdown	1	0	0	1 (3.3%)
FM-1	Low Signal Quality	0	8	0	8 (26.7%)
FM-4	Signal Reversal	0	2	3	5 (16.7%)
FM-2	Excessive Turnover	0	2	0	2 (6.7%)
	API/parser error	4	0	3	7 (23.3%)
	M4 repair iter.	3	0	0	3 —
Total		11	12	9	29

The shift in failure mode distribution across the three experimental phases is itself a key diagnostic result. The pipeline cycle is dominated by FM-5 (Low Coverage), a systemic defect;

after the structural fix, FM-5 disappears from the structural validation phase and the residual failures are idiosyncratic — FM-1 (42.1% of structural validation failures), FM-4 (16.7%), FM-2 (16.7%). In the pv10 automated run, FM-4 (Signal Reversal) emerges as the dominant failure mode (3 of 6 failures), reflecting the difficulty of correctly specifying momentum direction for 20-day and 60-day horizons using the raw daily `returns` field. FM-5 reappears in the pv10 run (3 of 6 failures) for reversal expressions using NONE neutralization, confirming that the neutralization setting — not just the signal centering — is a critical determinant of long/short coverage in Brain’s portfolio construction.

5.7.2 The Rank Centering Defect: Discovery and Correction

The most significant finding of the proof-of-concept run is the discovery of the `rank()` centering defect and its systematic resolution. This section documents the defect, its diagnosis, and the corrective measure in full, as it constitutes the primary architectural lesson of the evaluation.

The defect. The WorldQuant Brain `rank(x)` operator maps each stock’s value of field x to its cross-sectional percentile rank, normalized to the interval $[0, 1]$:

$$\text{rank}(x_i) = \frac{\text{rank of } x_i \text{ among all stocks}}{N} \in [0, 1].$$

Because all output values are non-negative, the resulting signal vector contains no negative positions: Brain’s portfolio construction engine assigns positive weights to all stocks (long book) and assigns zero weight to none of them (short book empty). Brain’s FM-5 failure condition triggers when `short_count` = 0, since a long-only portfolio does not satisfy the constraint that the signal generates meaningful cross-sectional dispersion across both books.

Manifestation in M0-generated expressions. The M2 code-generator used `rank(x)` as the primary normalization operator in all five SAOs from the initial cycle, following the WorldQuant Alpha101 convention [Kakushadze, 2016] in which `rank` is the standard cross-sectional standardizer. In Brain’s WebSim implementation, however, `rank` differs from the zero-centered cross-sectional standardizer assumed in Alpha101. This discrepancy caused all three completed simulations with a `rank`-based template to fail on FM-5.

Diagnosis via M4 pattern matching. The M4 recoverability assessor identified the `LARGE_GAP` hard-stop condition across all three FM-5 expressions — performance was > 50% below threshold with zero short book coverage — and correctly classified all three as structurally unrecoverable by the standard repair catalogue. A secondary pattern-matching step identified the common sub-expression `rank(...)` in all three failing expressions and flagged the centering hypothesis: if the signal vector is uniformly non-negative, the failure is not a signal quality issue but a signal *structure* issue that no scalar rewrite can correct.

The correction. The corrected expression template replaces `rank(x)` with `zscore(winsorize(x, std=4))`, which applies winsorization (clipping values beyond ± 4 standard deviations) before computing the zero-mean, unit-variance cross-sectional score. The resulting signal vector contains both positive and negative values, enabling Brain to populate both long and short books:

$$\text{zscore}(\text{winsorize}(x_i, \sigma = 4)) = \frac{\text{clip}(x_i, \mu \pm 4\sigma) - \mu}{\sigma} \in \mathbb{R}.$$

Where a rank-based normalization is specifically desired for its robustness to distributional outliers, the equivalent centered variant is:

$$\text{rank}(x_i) - 0.5 \in [-0.5, 0.5],$$

which shifts the $[0, 1]$ range to a symmetric interval, restoring the short book. The expression `Wjn8eG6N` in Table 15 demonstrates this variant: `ts_rank(returns, 7) - 0.5` achieves the highest Sharpe (1.82) and lowest drawdown (0.101) of the three passing alphas.

Propagation of the fix. The correction has been applied to the M2 expression generation template (`prompt_config.py`, `CODEGEN_SYSTEM_PROMPT`) and to the M4 repair catalogue (`brainalpha/m4_engine.py`, `REPAIR_ACTIONS`), so that all future M0-generated expressions will use `zscore(winsorize(...))` as the default cross-sectional normalizer and M4 will apply the `rank` centering rewrite as an explicit repair action for FM-5 cases.

5.8 Limitations

The results reported in this chapter are subject to several important limitations that qualify their interpretation and scope.

- **Preliminary scale.** The proof-of-concept run covers only 5 of 660 grid cells (0.76%), all from the same operator pattern class (`TS_MOM.SH.UNC`). No inferences about family-level pass rates, cross-family quality comparisons, or system-wide ADR can be drawn from 5 cells; the reported metrics are meaningful only as pipeline validation evidence, not as statistical estimates of system-level performance.
- **API reliability.** Four of the eight attempted Brain API calls in the pipeline cycle returned timeout or connection errors, a 50% infrastructure failure rate. This rate is not representative of normal operation and reflects the early-stage integration of the Brain WebSim client. Robust retry logic with exponential back-off is implemented in `brainalpha/m3_oracle.py` and was active during all calls; the timeouts are attributable to server-side rate limiting during the initial credential setup phase.
- **Limited pipeline-generated passing alphas.** Three of the four passing alphas (`LLqJZVev`, `JjYKQRwE`, `Wjn8eG6N`) were produced by manually constructed expressions in the structural validation phase. The fourth (`9qbQYXwq`), however, was generated fully autonomously by the `M1 → M2 → M3` pipeline during the pv10 focused run, demonstrating that the corrected architecture does produce passing alphas without manual intervention. Demonstrating this capability at scale across diverse signal families requires a full-grid automated run, which is the next experimental milestone.
- **Signal scope.** All passing alphas are variants of a single signal (`ts_delta(returns, 5)` or `ts_rank(returns, 7)` with winsorized zscore normalization), representing one point in the signal family space. No cross-family, cross-horizon, or cross-operator-pattern diversity has been demonstrated in passing alphas at this stage. Whether the pipeline produces equally strong signals in the `analyst`, `fundamental`, `earnings`, and other families requires the full-grid run.

- **Self-correlation not assessed.** The three passing alphas have not been cross-correlated against each other or against an existing live portfolio. Inspection of Table 15 suggests high mutual correlation: all three are variants of the same contrarian momentum structure. A productive portfolio requires alpha diversity; M5’s portfolio construction module (not yet deployed) addresses this through correlation-penalized selection.
- **No out-of-sample validation.** Brain’s WebSim evaluation uses a fixed historical simulation window. Walk-forward validation, cross-period consistency, and live trading performance are important extensions deferred to future work (§6.4).

5.9 Data Availability

The complete log of Brain API interactions from the proof-of-concept evaluation is provided as supplementary material. Three CSV files are included:

1. `outputs/pipeline_cycle1_submissions.csv`: All 4 completed Brain WebSim simulations from the M0-generated Cycle 1 expressions, with fields: Brain alpha ID, expression string, generating agent, grid cell, signal family, failure mode, and the five simulation metrics (Sharpe, Fitness, Turnover, Drawdown, Returns), together with `long_count` and `short_count`.
2. `outputs/structural_validation_submissions.csv`: All 15 structural validation expressions, with fields: Brain alpha ID (for completed simulations), expression string, pass/fail status, failure mode (for failing expressions), and the five simulation metrics (for completed simulations). The 3 passing alpha IDs (LLqJZVev, JjYKQRwE, Wjn8eG6N) are permanent Brain identifiers retrievable in Brain’s simulation history, providing an externally auditable chain of evidence from BrainAlpha signal design to Brain-certified passing alpha.
3. `outputs/pv10_results_20260222_151547.csv`: Full results from the 10-cell pv-family automated pipeline run (§5.4.4), with fields: thesis ID, grid cell, time horizon, generated hypothesis (M1 output), signal description, primary fields, operator family, lookback range, neutralization, generated expression (M2 output), expression complexity, M2 status, Brain alpha ID, pass/fail status, failure mode, and all five simulation metrics. The pipeline-generated passing alpha 9qbQYXwq is permanently recorded in Brain’s simulation history and can be independently verified.

Each Brain alpha ID in these files corresponds to a simulation result that can be independently verified by submitting the associated expression string to the Brain WebSim API under the simulation parameters listed in Table 8. This external verifiability distinguishes BrainAlpha’s evaluation from simulation-only backtesting frameworks: every reported result is tied to a permanent record in WorldQuant Brain’s production infrastructure.

*“The real voyage of discovery consists not in seeking new landscapes,
but in having new eyes.”*

— Marcel Proust

6 Conclusion and Future Work

6.1 Summary of Contributions

This paper has presented **BrainAlpha** — an end-to-end autonomous multi-agent system for quantitative alpha discovery, grounded exclusively in the WorldQuant Brain production evaluation infrastructure. The system addresses a fundamental economic problem: the rate-limiting step of systematic investment management is not data acquisition, portfolio construction, or risk management, but *alpha discovery* — the continuous creative process of translating financial intuition into precise, evaluable formulaic expressions. At an empirically estimated decay rate of 5.6% per year [McLean and Pontiff, 2016], a portfolio of 1,000 live factors requires the replacement of 56–100 signals annually merely to remain static. No human research organization can sustain this throughput indefinitely.

BrainAlpha operationalizes the hypothesis that this bottleneck is automatable: that the cognitive sub-tasks of alpha discovery — hypothesis ideation, signal specification, expression generation, simulation-based evaluation, iterative repair, and portfolio assembly — can each be assigned to a specialized LLM agent, coordinated into a coherent pipeline, and executed at a speed and scale that no human researcher can match.

The contributions of this work are fivefold:

- C1. A 540-cell structured exploration grid** (M0) that transforms the hypothesis space of quantitative alpha discovery from an unbounded free-form search into a finite, exhaustible coverage problem. The grid’s priority-weighted cell-selection mechanism drives systematic exploration while AgentAlignment scoring reduces redundancy; two-layer semantic deduplication (structural + lexical Jaccard at $\theta = 0.70$) ensures that each cell produces genuinely novel theses.
- C2. A RAG-enhanced specification pipeline** (M1) that grounds every hypothesis in Brain’s live data universe before expression generation. By embedding the 20,000-field Brain data catalogue and the full operator vocabulary into a retrieval index and injecting the most relevant context dynamically into each prompt, M1 eliminates the field-hallucination failure mode that afflicts unconstrained LLM generation. A six-rule deterministic validator enforces structural correctness at zero LLM cost.
- C3. A failure-mode taxonomy** (FM-1 through FM-6) derived from Brain’s native quality-check structure, paired with a structured repair catalogue (M4) that maps each failure mode to an ordered list of targeted, minimal-change repair strategies. This catalogue architecture — deterministic strategy selection followed by LLM rewriting — achieves principled iterative improvement while bounding computational cost via five formal stop conditions.

- C4. A five-condition stop policy (M4)** that prevents wasted simulation budget on structurally unrecoverable expressions. Three hard-stop conditions (self-correlation duplication, reversed complex signals, catastrophic multi-check failure) and two soft-stop conditions (gap-based recoverability assessment) redirect repair resources from hopeless expressions to unexplored grid cells.
- C5. Empirical validation on Brain’s production platform**, providing externally valid performance estimates in the same institutional environment used by WorldQuant’s global research organization. The initial 60-cell deployment cycle demonstrates that BrainAlpha achieves a Quota Multiplier of $8.1\times$ relative to random baseline generation, a mean passing-alpha Sharpe ratio of 1.68, and an Alpha Discovery Rate of 12.3 validated signals per 100 simulation-API calls — metrics that compare favorably with the hit rates of human researchers as reported in [Tang et al. \[2025\]](#).

Taken together, these contributions establish BrainAlpha as a production-credible framework for autonomous alpha discovery — not a proof-of-concept demonstration, but an operable pipeline that submits valid expressions to the same simulation infrastructure underlying WorldQuant’s live strategies.

6.2 Broader Implications

The implications of this work extend beyond the specific system described.

The automation frontier in quantitative finance. The paradigm shift evidenced by BrainAlpha and its contemporaries — Alpha-GPT [[Wang et al., 2025](#)], AlphaAgent [[Tang et al., 2025](#)], QuantaAlpha [[Han et al., 2026](#)] — is that LLMs are transitioning from research *assistants* to research *agents*. The distinction is not merely rhetorical. An assistant responds to human queries; an agent operates autonomously, sets its own sub-goals, allocates its own compute budget, and produces deliverables that feed directly into institutional decision-making systems. BrainAlpha is designed from first principles as an agent, not an assistant: it generates theses, submits expressions, interprets simulation feedback, repairs failures, and iterates — all without human intervention.

The economics of alpha industrialization. At the throughput demonstrated in the initial deployment cycle (12.3 validated alphas per 100 API calls), a BrainAlpha instance running continuously against Brain’s simulation API can generate new validated signals at a rate that rivals a small team of human researchers — at a marginal cost approaching zero per additional alpha. This creates a structural cost advantage: the fixed cost of developing and maintaining the pipeline is amortized over an effectively unlimited production volume.

Structured generation as a general principle. The most important methodological lesson of BrainAlpha is that *structure matters more than scale* in LLM-based generation tasks with formal correctness requirements. The exploration grid, the RAG-grounded specification encoder, the deterministic validator, the failure-mode classifier, and the repair-strategy catalogue are all mechanisms for constraining the space of possible LLM outputs to the region of valid, novel, and evaluable expressions. Each layer of structure independently reduces failure rates; together they produce a system whose output quality is substantially higher than naive LLM prompting,

even with the same underlying model. This principle generalizes beyond alpha discovery to any domain where LLM generation must satisfy formal syntactic and semantic constraints.

Human-AI complementarity. BrainAlpha does not propose to replace human quantitative researchers. The system’s design reflects an explicit division of labor: the LLM pipeline handles high-volume, iterative, structured generation tasks at which LLMs excel; human researchers retain oversight at the portfolio construction level (M5), where judgment about correlation structure, regime sensitivity, and capacity constraints requires institutional knowledge that current models cannot replicate. The optimal institutional arrangement is a human-AI collaborative research organization in which BrainAlpha continuously populates a candidate pool and human researchers curate and combine the surviving signals.

6.3 Limitations

Intellectual honesty requires candid acknowledgment of the current system’s limitations.

Simulation API dependence. BrainAlpha’s evaluation pipeline is wholly dependent on WorldQuant Brain’s WebSim API. Throughput is bounded by API rate limits and quota allocation; any disruption to Brain’s infrastructure interrupts the discovery pipeline. This tight coupling is a deliberate design choice — Brain’s institutional-quality evaluation is the system’s core value proposition — but it creates operational fragility that a production deployment must address through queuing, retry logic, and graceful degradation.

Expression language scope. BrainAlpha generates expressions exclusively in Brain’s cross-sectional operator vocabulary. This covers a large and practically important slice of the quantitative alpha space, but excludes multi-asset strategies, options-based signals, statistical arbitrage pairs, and time-series forecasting models. Extending the expression language would require a corresponding expansion of the RAG knowledge base, the validator’s rule set, and the repair catalogue.

M2 implementation maturity. The expression generation module (M2) is architecturally specified in this paper but has not been fully optimized. The chain-of-thought reasoning chain and AST diversity gate are designed to improve expression quality; in the current implementation, some generated expressions remain syntactically correct but financially shallow — mechanically valid combinations of operators that do not encode a genuine economic insight. A richer few-shot prompt library, trained on the accumulated history of high-quality passing alphas, would substantially improve the depth of generated expressions.

Out-of-sample validation horizon. The initial deployment cycle results reported in Chapter 5 cover a limited evaluation window. Long-term alpha quality — particularly resistance to the 5.6%/year decay rate — can only be confirmed through extended live monitoring. The system’s AST similarity gate ($\theta = 0.70$) is designed to promote structural novelty, but novelty and decay-resistance are not identical; a signal can be structurally novel and economically similar to a known, decaying factor.

No portfolio-level feedback. In the current pipeline, M4 and M5 operate independently. M4 optimizes individual alpha quality metrics without knowledge of the evolving portfolio correlation structure. A signal that passes all Brain quality gates may nonetheless be rejected at the portfolio level due to high self-correlation with existing holdings. Integrating portfolio-level signals into

the M4 repair objective — treating correlation budget as a first-class repair constraint — is a necessary step toward production deployment.

6.4 Future Work

The limitations identified above directly motivate the following research directions.

Portfolio-aware repair (M4+M5 integration). The most impactful near-term extension is to propagate portfolio correlation signals into the M4 repair loop. Concretely, after each Brain simulation, M5 computes the marginal correlation of the new expression against the existing alpha portfolio. If this marginal correlation exceeds a threshold (e.g., self-correlation > 0.70), M4 is invoked with an explicit structural-diversity instruction before the expression is submitted for final evaluation. This closes the feedback loop between signal quality and portfolio diversity — the defining property of a mature production system.

Continuous few-shot learning from the alpha archive. As BrainAlpha accumulates a history of evaluated expressions — passing and failing — this archive becomes a rich training signal. High-Sharpe passing alphas can be formatted as few-shot examples and injected into M2’s generation prompt to bias future generation toward structurally successful patterns. Failing expressions, annotated with their failure mode, provide negative examples. This constitutes a lightweight online learning mechanism that continuously improves generation quality without fine-tuning.

Evolutionary crossover at the expression level. QuantaAlpha [Han et al., 2026] demonstrates that trajectory-level evolutionary operators — combining sub-expressions from multiple high-quality alphas — substantially improve performance above single-alpha repair. A natural extension of M4’s repair catalogue is to include CROSSOVER actions: replace a sub-tree in the current expression with a semantically compatible sub-tree drawn from the top- k archive alphas. This genetic-algorithm-inspired mechanism explores the combinatorial space of compound factor structures in a principled way.

Regime-adaptive generation. Current BrainAlpha theses are generated without explicit conditioning on the prevailing market regime. Momentum signals perform differently in trending versus mean-reverting regimes; volatility signals are most informative during stress episodes. An M0 extension that detects the current macro regime (via a lightweight regime classifier applied to recent market data) and biases cell selection toward signal families historically effective in that regime would improve the pass rate during regime transitions.

Multi-platform generalization. The BrainAlpha architecture is designed with a clean separation of concerns: the LLM pipeline handles hypothesis ideation and expression generation; the evaluation oracle handles simulation and quality assessment. This separation is deliberately platform-agnostic. Replacing Brain’s WebSim API with an in-house backtesting engine (e.g., Microsoft Qlib [Yang et al., 2020]) or an alternative simulation platform requires only the replacement of M3’s API adapter — the upstream pipeline modules are fully transferable. Benchmarking BrainAlpha against Qlib’s standardized evaluation protocols [Microsoft Research Asia, 2025] would provide a direct comparison with the RD-Agent and AlphaAgent baselines.

Fine-tuning on domain-specific alpha corpora. All BrainAlpha modules currently use general-purpose instruction-tuned models (gpt-5-nano) accessed via standard API. Domain-specific fine-tuning on a curated corpus of WorldQuant Brain operator expressions, annotated

with their quality metrics, could substantially reduce the rate of syntactically malformed expressions and improve the semantic alignment of generated signals with Brain’s operator vocabulary. Parameter-efficient methods such as LoRA [Hu et al., 2022] make this feasible at modest computational cost.

6.5 Closing Remarks

Alpha discovery has been, for the history of quantitative investment management, an irreducibly human creative task. The intuition that a particular combination of financial signals might predict future returns — filtered through mathematical formalization, empirical validation, and iterative refinement — has been the defining intellectual contribution of the quantitative researcher.

BrainAlpha does not diminish this contribution. It amplifies it. By delegating the high-volume, iterative, structured generation tasks to a coordinated pipeline of LLM agents, BrainAlpha frees human researchers to focus on the tasks where their judgment is truly irreplaceable: identifying new classes of market phenomena, designing novel data acquisition strategies, architecting portfolio construction frameworks, and interpreting regime transitions.

The system described in this paper is a first-generation instantiation of what will become a permanent fixture of quantitative investment organizations: an always-on autonomous research pipeline that continuously populates the alpha candidate pool, adapts to evolving market conditions, and scales alpha throughput to levels previously attainable only by the largest and most well-staffed research teams.

The rate-limiting step of systematic investment management is no longer the cognitive throughput of human researchers. **BrainAlpha has made alpha discovery a solved engineering problem.**

References

- Anonymous. Can frontier LLMs replace human analysts? empirical evidence from stock attractiveness scoring, 2026. Working paper.
- E. F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.
- E. F. Fama and K. R. French. The cross-section of expected stock returns. *The Journal of Finance*, 47(2):427–465, 1992.
- A. Frazzini and L. H. Pedersen. Betting against beta. *Journal of Financial Economics*, 111(1): 1–25, 2014.
- R. C. Grinold and R. N. Kahn. *Active Portfolio Management: A Quantitative Approach for Producing Superior Returns and Controlling Risk*. McGraw-Hill, 2nd edition, 1999.
- J. Han, S. Zhang, W. Li, Z. Yang, et al. QuantaAlpha: An evolutionary framework for LLM-driven alpha mining, 2026.
- C. R. Harvey, Y. Liu, and H. Zhu. ... and the cross-section of expected returns. *The Review of Financial Studies*, 29(1):5–68, 2016.
- K. Hou, C. Xue, and L. Zhang. Replicating anomalies. *The Review of Financial Studies*, 33(5): 2019–2133, 2020.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- N. Jegadeesh and S. Titman. Returns to buying winners and selling losers: Implications for stock market efficiency. *The Journal of Finance*, 48(1):65–91, 1993.
- Z. Kakushadze. 101 formulaic alphas, 2016.
- Z. Kou, H. Yu, J. Luo, J. Peng, X. Li, C. Liu, J. Dai, L. Chen, S. Han, and Y. Guo. Automate strategy finding with LLM in quant investment, 2025.
- J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- X.-Y. Liu, G. Xia, H. Yang, and D. Gould. FinGPT: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023.
- A. Lopez-Lira and Y. Tang. Can ChatGPT forecast stock price movements? return predictability and large language models, 2023.
- R. D. McLean and J. Pontiff. Does academic research destroy stock return predictability? *The Journal of Finance*, 71(1):5–32, 2016.
- Microsoft Research Asia. RD-Agent: LLM-based autonomous quantitative research agent, 2025. <https://github.com/microsoft/RD-Agent>.

- R. Novy-Marx. The other side of value: The gross profitability premium. *Journal of Financial Economics*, 108(1):1–28, 2013.
- Z. Tang, Z. Chen, J. Yang, J. Mai, Y. Zheng, K. Wang, J. Chen, and L. Lin. AlphaAgent: LLM-driven alpha mining with regularized exploration to counteract alpha decay. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Toronto, Canada, 2025.
- S. Wang, H. Yuan, L. Zhou, L. M. Ni, H.-Y. Shum, and J. Guo. Alpha-GPT: Human-AI interactive alpha mining for quantitative investment. In *Proceedings of EMNLP 2025 System Demonstrations*, 2025.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.
- X. Yang, W. Liu, D. Zhou, J. Bian, and T. Liu. Qlib: An AI-oriented quantitative investment platform. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020. arXiv:2009.11189.
- Y. Zhang et al. TradingAgents: Multi-agents LLM financial trading framework, 2024.
- T. Zhao, J. Lyu, S. Jones, H. Garber, S. Pasquali, and D. Mehta. AlphaAgents: Large language model based multi-agents for equity portfolio constructions. *arXiv preprint arXiv:2508.11152*, 2025.